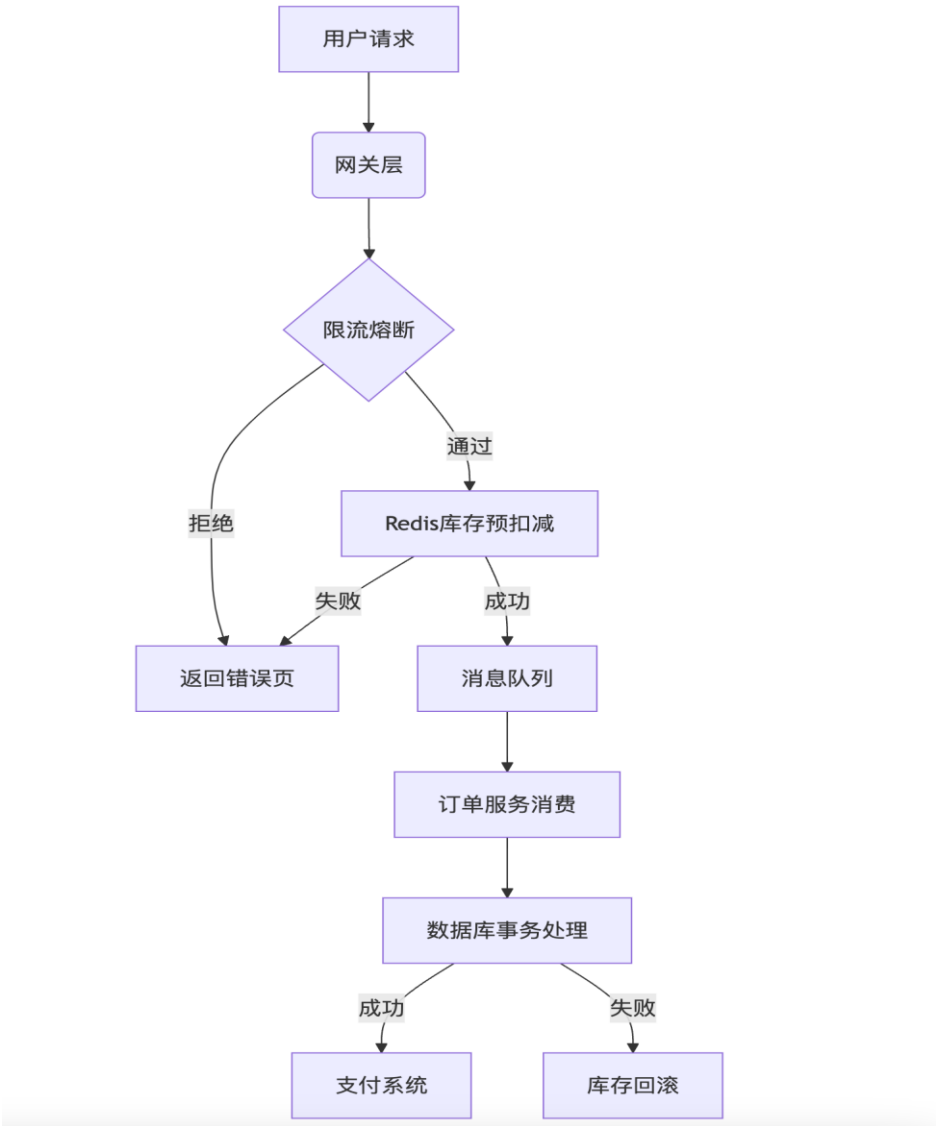
系统设计面试题

如何设计一个秒杀场景?

秒杀场景的核心特点是**高并发、低库存、短时间爆发式访问** 。因此，设计时需要解决以下几个问题：

1. **高并发处理** ：如何应对大量用户同时访问？
2. **库存一致性** ：如何保证库存不会超卖或少卖？
3. **用户体验** ：如何减少用户等待时间，避免页面崩溃？
4. **防刷机制** ：如何防止恶意用户利用脚本抢购商品？

面对上面这些问题，可以针对每一层做一些设计：



1、前端层：

- **静态资源分离** ：将秒杀页面的静态资源（如 HTML、CSS、JS）部署到 CDN（内容分发网络），减轻服务器压力。
- **请求拦截**：活动未开始时，前端按钮置灰；通过验证码、点击频率限制。

2、网关层：

- **流量拦截** ：使用 API 网关对请求进行初步过滤，例如 IP 限流、黑名单拦截等。
- **身份验证** ：通过 Token 或签名验证用户身份，防止未登录用户直接访问秒杀接口。

3、缓存层：

- **Redis 缓存库存** ：将商品库存信息存储在 Redis 中，利用其高性能特性处理库存扣减操作。
- **预热数据** ：在秒杀活动开始前，将商品信息和库存数据加载到 Redis 中，减少数据库压力。

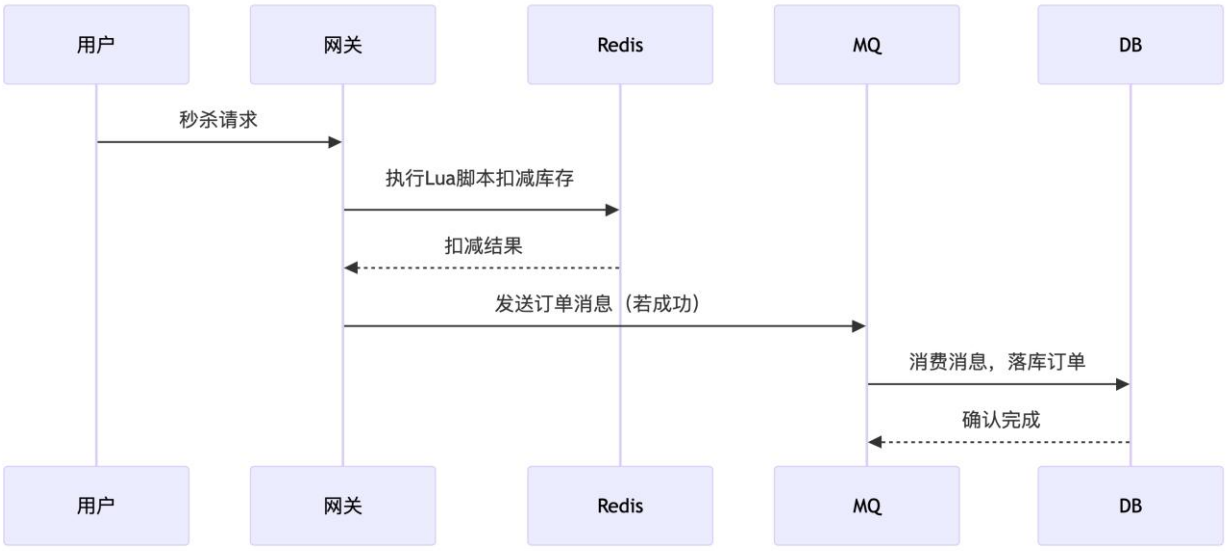
4、消息队列：

- **削峰填谷** ：使用消息队列（如 Kafka）将用户的秒杀请求异步化，避免直接冲击后端服务。
- **订单处理** ：将成功的秒杀请求放入队列，由后台服务异步生成订单，提高系统吞吐量。

5、数据库层

- **乐观锁**：在库存扣减时，使用乐观锁确保库存一致性。
- **读写分离** ：通过主从复制实现数据库的读写分离，提升查询性能。

关键的核心业务逻辑实现，**库存防超卖方案采用：Redis 原子操作 + 异步扣减数据库**。



具体流程如下：

- **秒杀请求达到**：用户发起秒杀请求，系统接收到请求后，首先进行一些基础校验（如用户身份验证、活动是否开始等）。如果校验通过，进入库存扣减逻辑。
- **Redis 库存扣减**：在 Redis 中检查商品库存是否充足。例如，使用 `GET` 命令获取当前库存数量。如果库存不足，直接返回失败，结束流程。如果库存充足，使用 Redis 的原子操作（如 `DECR` 或 Lua 脚本）扣减库存。
- **异步更新数据库**：如果 Redis 库存扣减成功，生成一个秒杀成功的消息，并将其放入消息队列
- **后台服务消费消息**：后台服务从消息队列中消费秒杀成功的消息，执行以下操作：
    - 1、为用户创建订单记录；
    - 2、使用乐观锁将数据库中的库存数量减少 1；
    - 3、通过唯一标识（如用户 ID+商品 ID+时间戳）防止重复消费。
- **最终一致性校验**：在 Redis 库存扣减和数据库库存更新之间，可能会存在短暂的不一致状态。为了保证最终一致性，可以采取以下措施：
    - 1、定期将 Redis 中的库存数据与数据库进行同步。
    - 2、如果发现 Redis 和数据库库存不一致，触发补偿逻辑（如回滚订单或调整库存）。

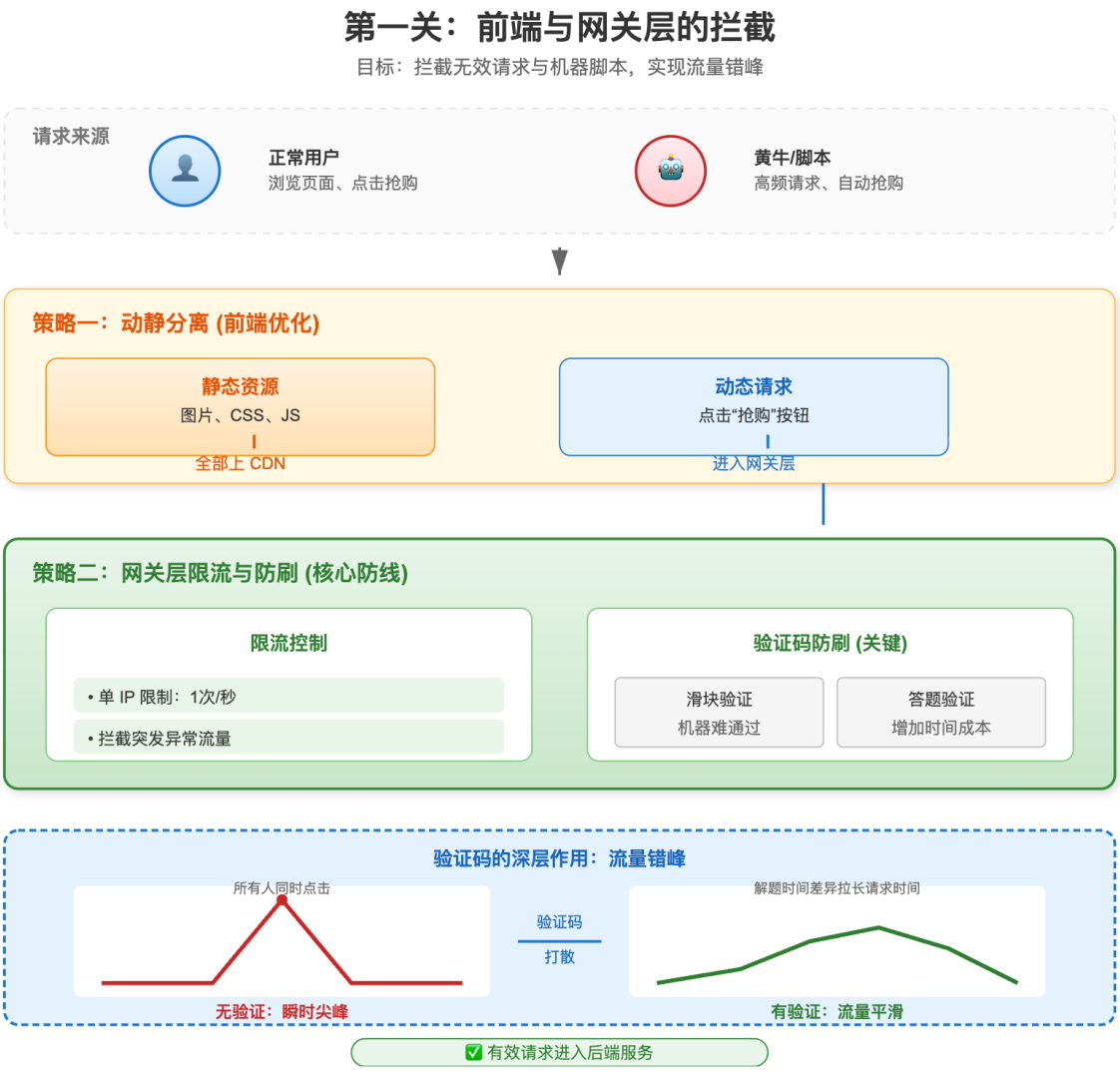
> 回答参考

关于秒杀系统的设计，其实业界有一个非常经典的共识，那就是「**层层漏斗**」原则。

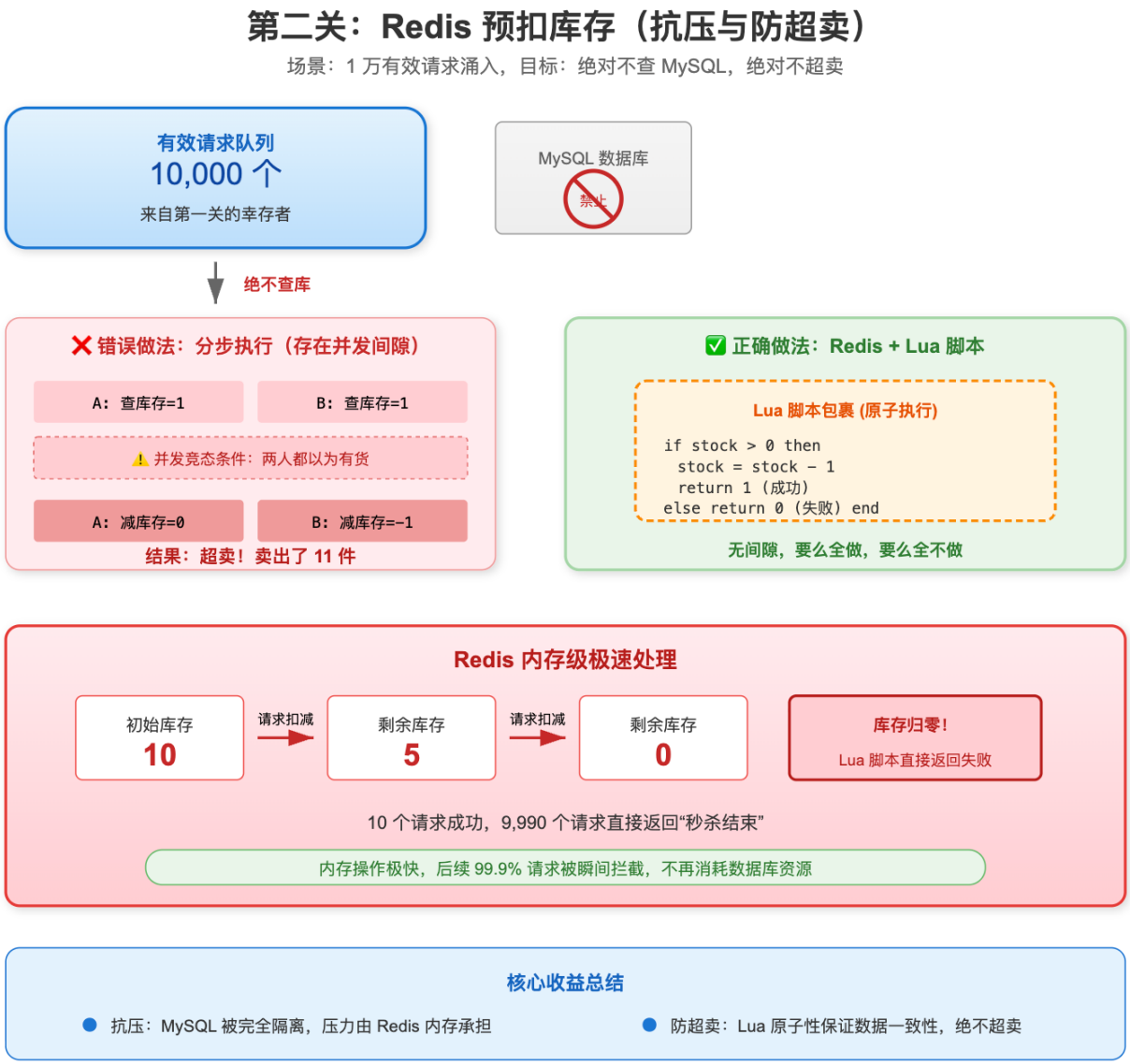
秒杀场景最大的特点就是「瞬时并发极高、读多写少、且绝对不能超卖」。比如 10 万个人抢 10 台手机，如果让这 10 万个请求直接打到 MySQL 数据库里，肯定瞬间就宕机了。所以我的大体设计思路是：**在请求到达最终的数据库之前，设置多道防线，层层拦截，尽量把流量挡在上游。**

具体落地的话，我会从以下三个关卡来汇报我的设计过程：

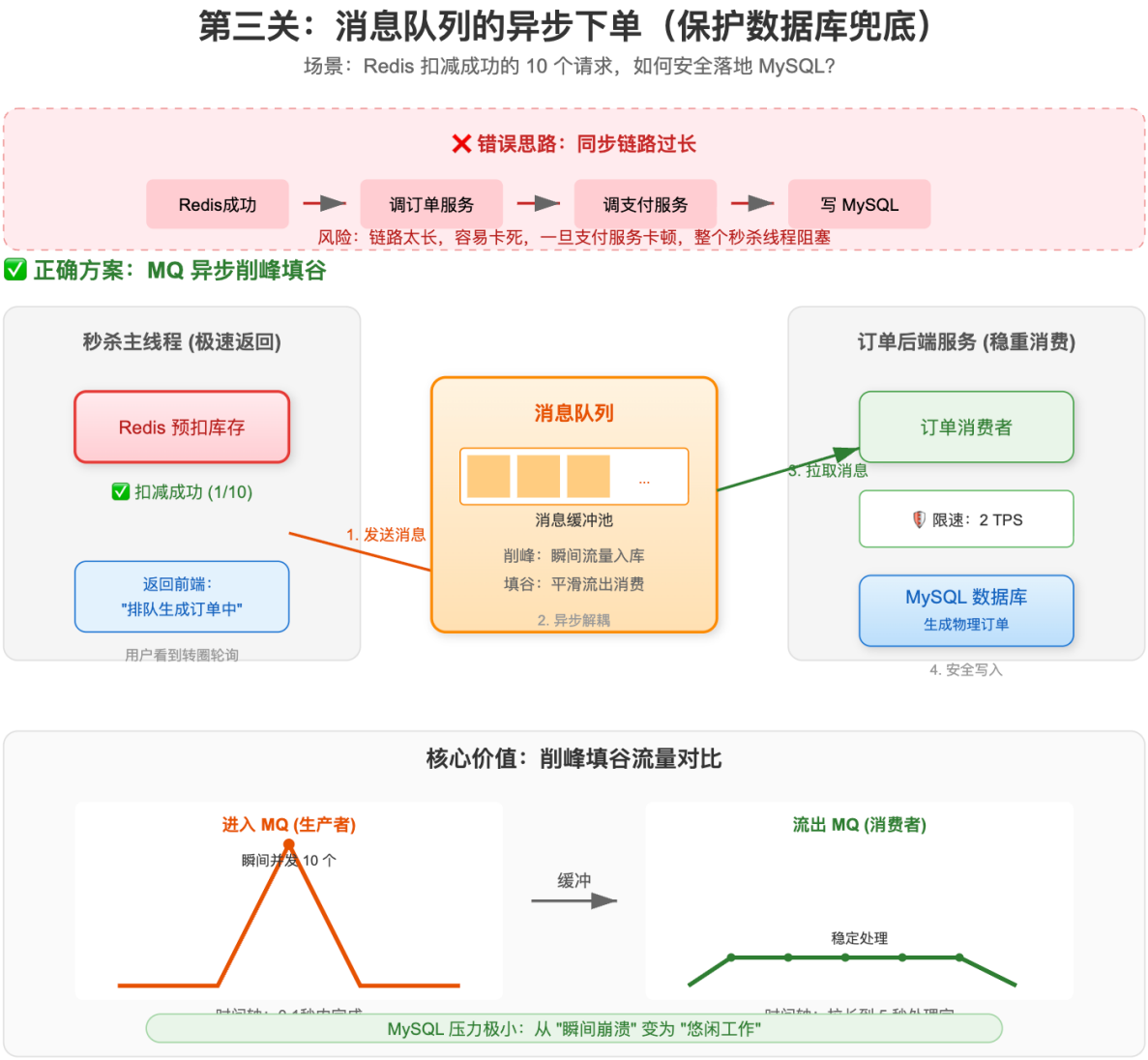
第一关：前端与网关层的拦截（动静分离与防刷） 在这个阶段我们的目的是尽可能拦住「无效请求」和「机器脚本」。 首先，秒杀页面的所有图片、CSS 等静态资源全部要上 CDN，做「**动静分离**」，只有点「抢购」那个瞬间产生的是动态请求。 其次，网关层必须做**限流和防刷**。比如同一个单 IP 一秒只能请求 1 次，或者在抢购前加一个「滑块验证码」或「答题」。加验证码这招非常管用，一方面能防住大量黄牛的脚本机器，另一方面把用户原本高度集中的一毫秒点击，在物理上打散成了好几秒（因为大家算题时间不一样），起到了第一层的流量错峰作用。



第二关：真正的秒杀核心，Redis 预扣库存（抗压与防超卖） 经过第一关，假设还有 1 万个有效请求进来了，这时候绝对不能去查 MySQL。所有的库存必须提前缓存进 Redis。 这里的重点是**如何防止超卖**。很多人以为查一下 Redis 还有库存，然后再减去 1 就可以了。但在高并发下，这「一查一减」之间，别人可能插队进来了，导致卖出去了 11 件商品。 为了解决这个同步问题，我会用 **Redis 组合 Lua 脚本**。您可以把 Lua 脚本理解为一个小包裹，它能在 Redis 层把「查库存」和「扣库存」这两个动作捆绑在一起，原子性地去执行。只要 Redis 变成了 0，剩下的 9990 个请求全部直接返回「库存已空，秒杀结束」，绝不往下走。这样不仅性能极高，而且在内存层面绝对杜绝了超卖。



第三关：消息队列的异步下单（保护数据库兜底） 好了，现在 Redis 里刚刚成功扣减了 10 个库存。这幸运的 10 个人，我们是不是立马就在全链路里去调用订单服务、支付服务，最后把数据写进 MySQL 呢？ 依然不行。因为链路太长，容易卡死。这个时候我会引入 **消息队列（MQ）**做异步削峰。 Redis 扣减成功后，服务仅仅是往 MQ 发一条消息：「用户 A 抢到了手机」，然后直接返回给前端：「您已抢购成功，正在排队生成订单中」。前端页面这个时候开始转圈轮询。 在这背后，订单服务作为 MQ 的消费者，它不管外面风多大，只会根据数据库自己能承受的速度（比如一秒处理两个），慢条斯理地从 MQ 里把消息拉出来，真正在 MySQL 里扣除库存、生成物理订单。这也就是我们常说的「**异步削峰填谷**」。由于走到这一步的流量最多也就两位数或三位数，所以 MySQL 就非常安全了。



总结一下我的整体链路： 就是利用**动静分离和验证码**拦住第一波；然后让请求进入单机十万 QPS 的 Redis **通过 Lua 脚本**做原子预扣减拦住第二波；真正幸存的极少数流量，放入 **MQ 排队削峰**，最后才交给 MySQL 落地。这就是一套高性能且防超卖的秒杀设计。

让你设计一个 RPC 框架，怎么设计?

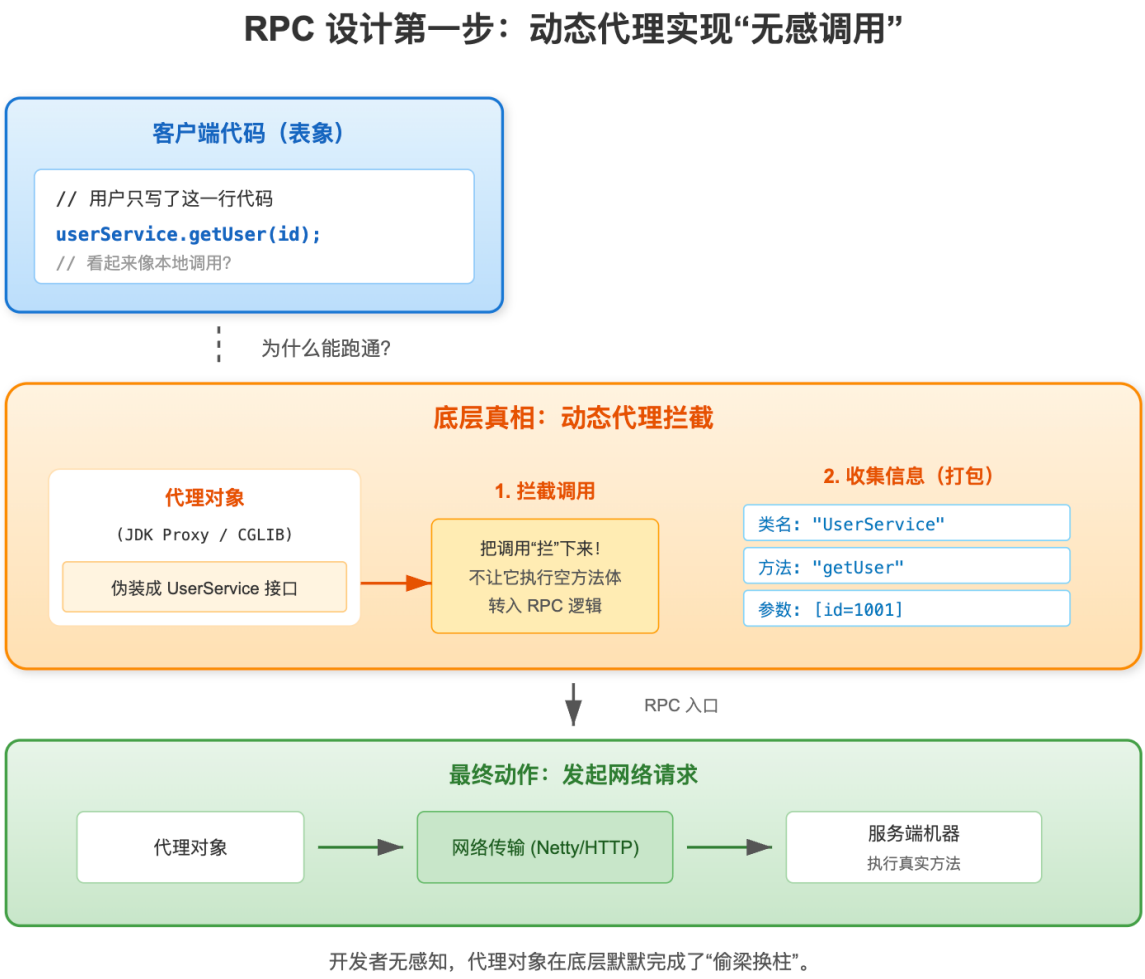
其实这就相当于要我们亲手去造一个类似 Dubbo 或者 gRPC 的轮子。

我是这样来思考和设计的，设计的核心目标只有一个：**让业务开发人员调用远端服务时，就像调用本地方法一样傻瓜、简单，完全不需要去关心底层网络的那些脏活累活。**

顺着一条请求从发出到接收的完整路程，我会把这个框架拆分成以下几个核心的模块来逐步设计：

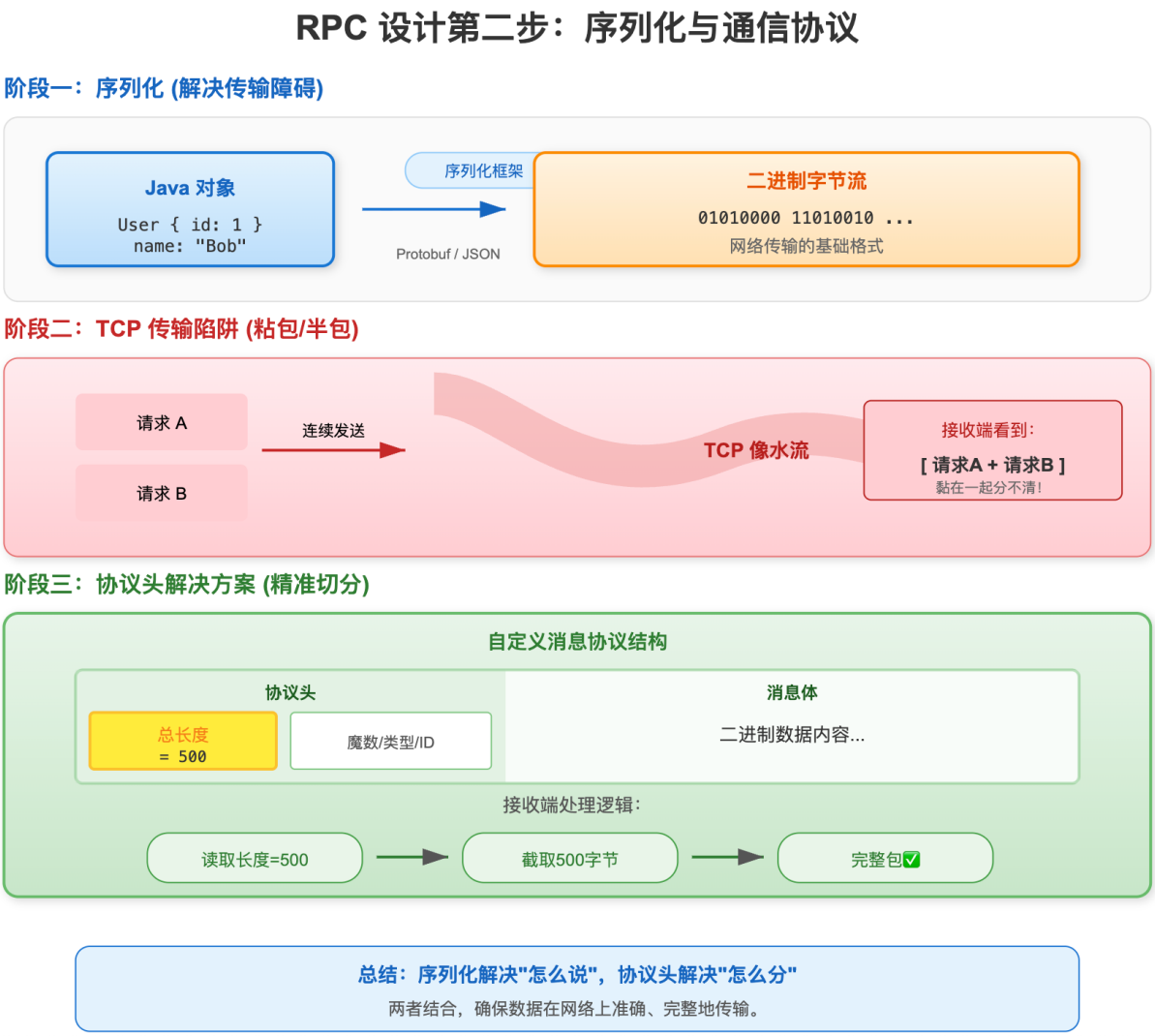
第一步是要解决「无感调用」的问题，这里需要引入：动态代理模式。 在代码里，客户端明明只是调了一个接口 `userService.getUser(id)`，它怎么就跑到另一台机器上去了呢？这就需要 RPC 框架在底层用动态代理（比如 JDK 自带的，或者 CGLIB）偷偷生成一个代理对象。这个代理会把用户的

调用给「拦截」下来，把我要调哪个类、哪个方法、传了什么参数这些信息统统收集起来。这就是整个 RPC 真正的入口。

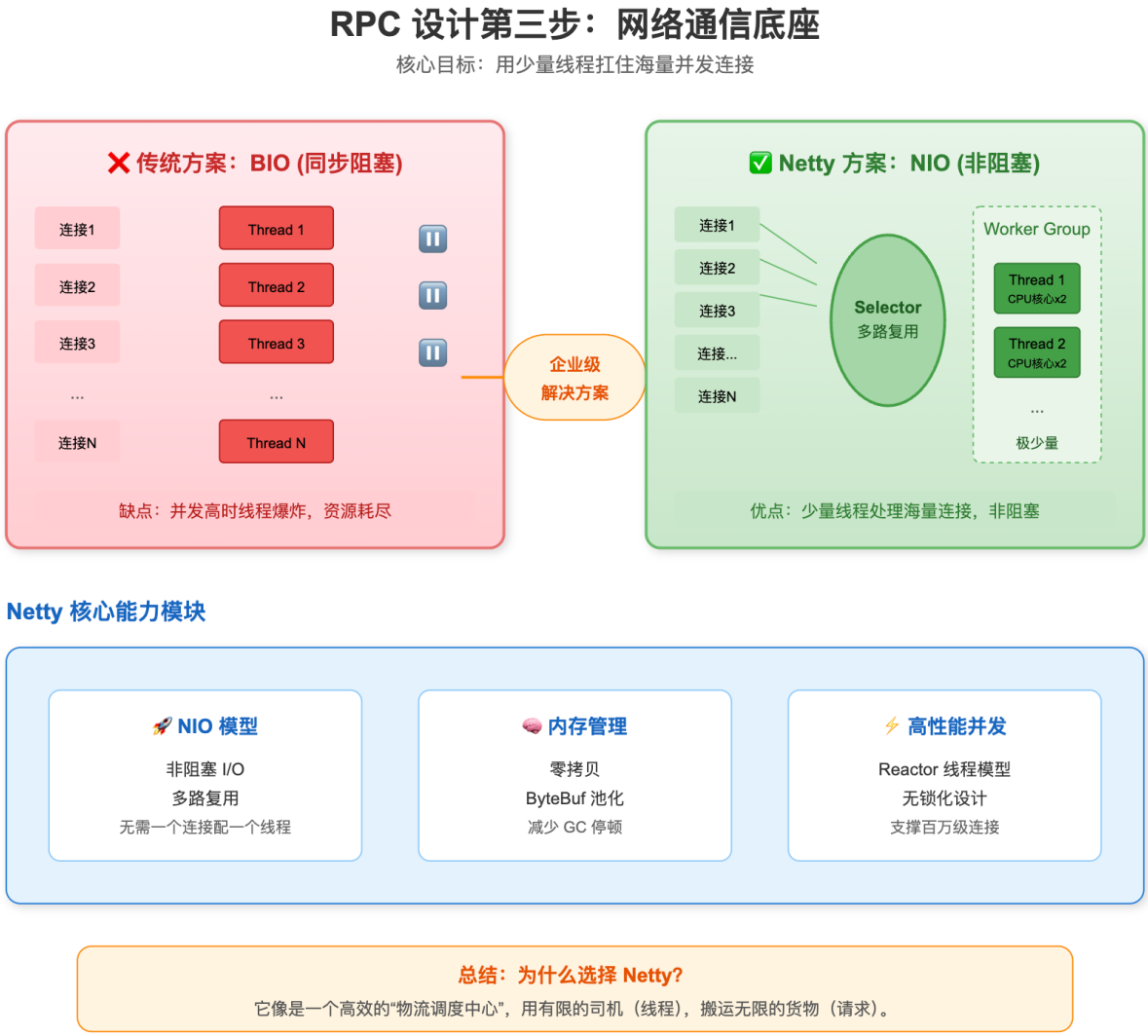


开发者无感知，代理对象在底层默默完成了“偷梁换柱”。

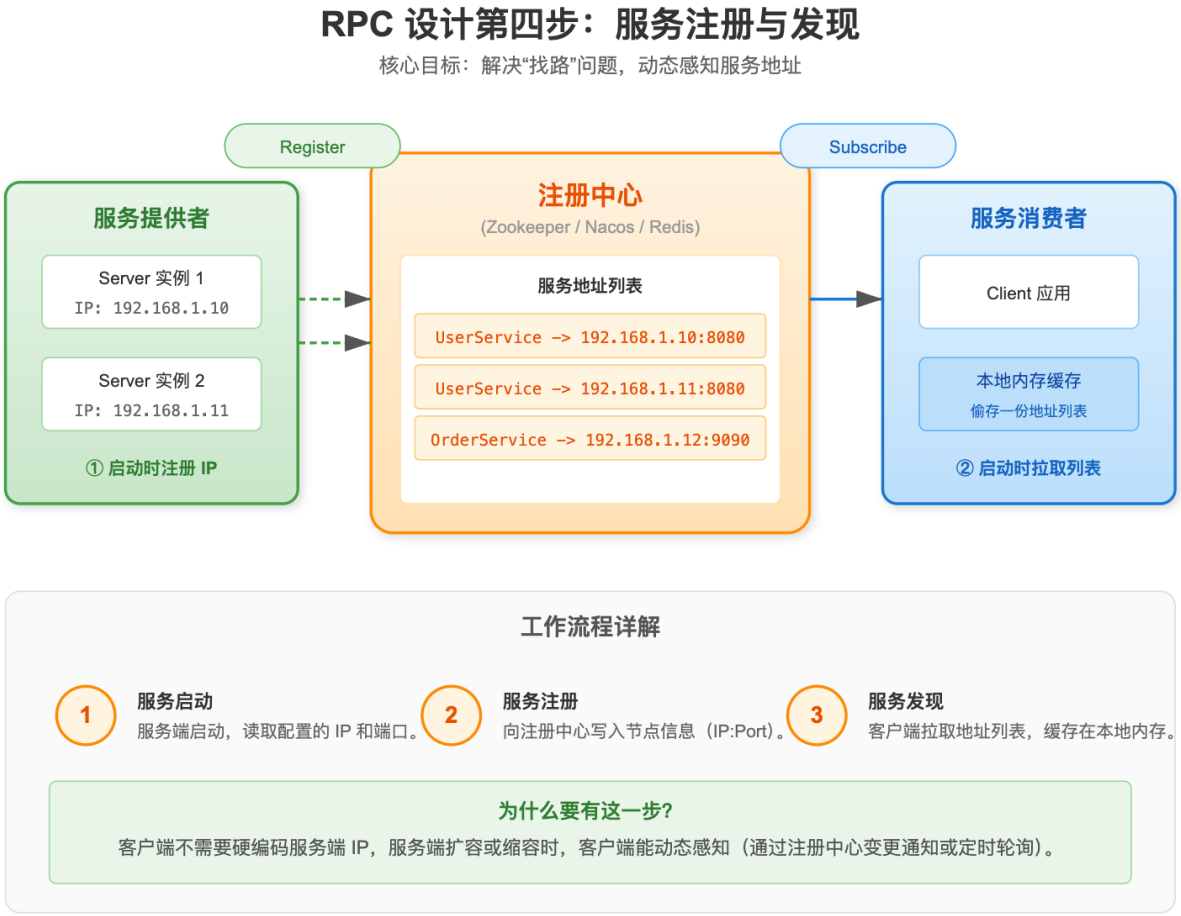
第二步是数据打包，这里需要设计：序列化与通信协议模块。 收集到参数后，网络线上是不能传输 Java 对象的，只能传二进制的 0101 字节流。所以我们需要引入**序列化框架**（比如 Protobuf、Hessian 或者 JSON），把对象压缩成字节数组。 但光有字节数据还不够，由于底层 TCP 传输是像水流一样的，如果我们连续发两个请求，接收端很容易黏在一起分不清谁是谁（这就是著名的 TCP 粘包/半包问题）。为了解决这个问题，我还要设计一个**自定义协议头**。在这个协议头里，我会明确写上「这段消息的总长度是多少」，这样接收端每次严格按照长度去切分数据，就不会乱了。



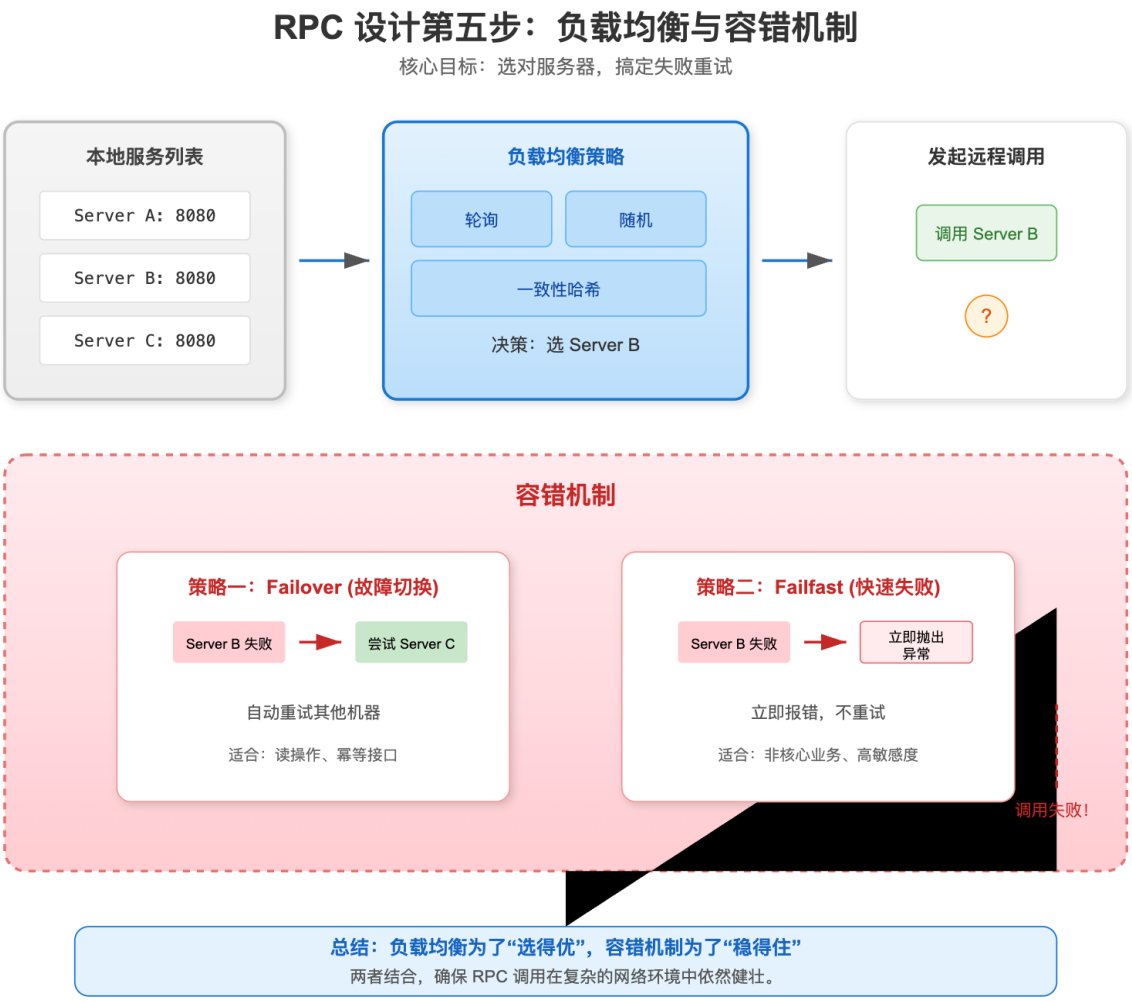
第三步是负责数据的高效搬运，也就是：网络通信底座。 数据准备好了，怎么发出去？考虑到企业级并发量，我绝对不会用传统的同步阻塞网络 IO，而是会引入 `Netty` 来作为底层的网络通信框架。利用 Netty 的 NIO（非阻塞 IO）模型和优秀的内存管理，用极少的线程就能处理海量的并发网络请求。



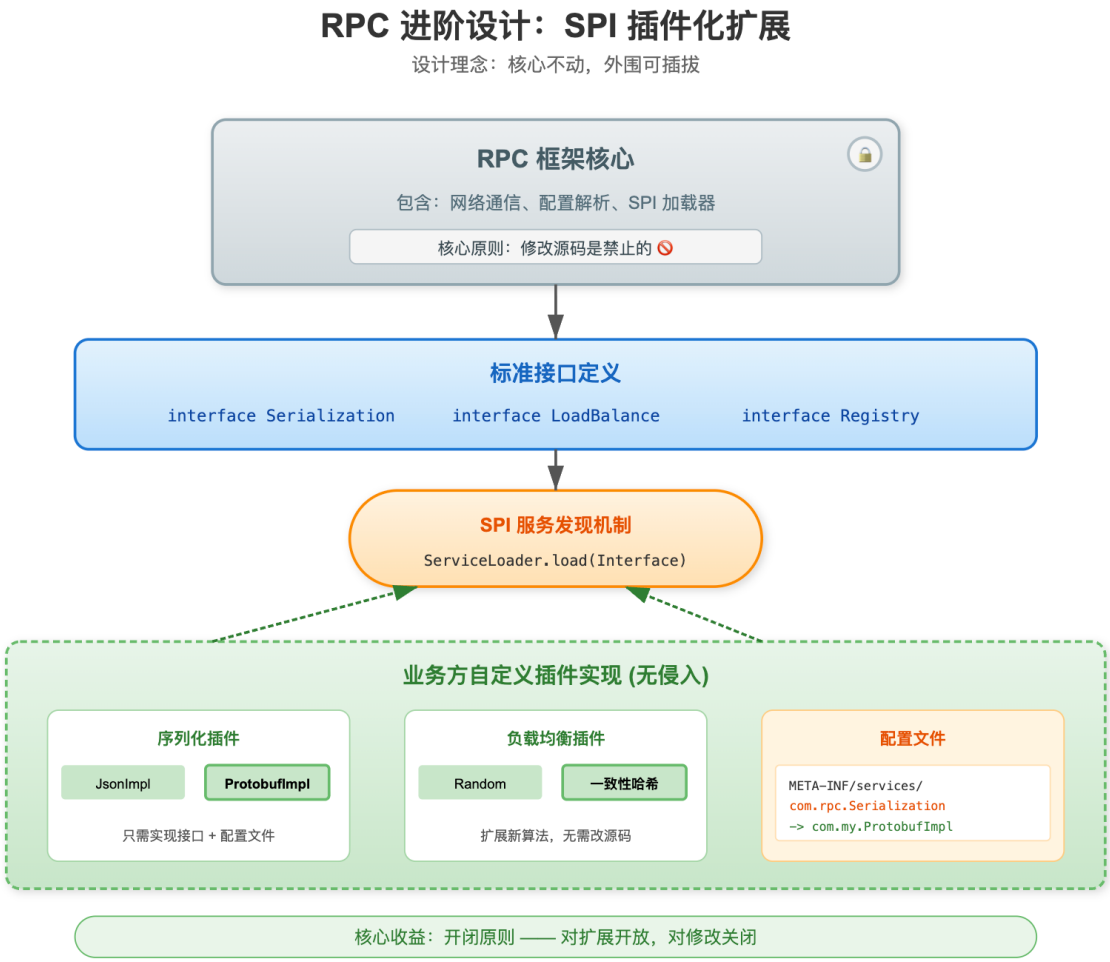
第四步是解决找路的问题，需要设计：服务注册与发现模块。 数据再好，不知道往哪台服务器发也是白搭。所以框架里必须对接一个**注册中心**（比如 Zookeeper、Nacos 或者 Redis）。 服务端一旦启动，就把自己的 IP 和端口挂载到注册中心；客户端启动时，就从注册中心拉取一份可用服务器的地址列表，偷偷缓存在本地内存里。



第五步是保障系统的稳定性，加入：负载均衡与容错机制。 客户端拿到了一大把服务端的 IP，总不能每次都选第一个吧？这时候就需要设计**负载均衡模块**，用比如「轮询」、「随机」或者「一致性哈希」的算法挑一台机器出来。 如果挑的这台机器刚好网络闪断、调用失败了怎么办？所以最后还要包装一层**容错策略**，比如配置成自动去重试另一台机器（Failover），或者快速失败并抛出异常（Failfast）。



最后我想补充一点进阶的设计思路：扩展性。 一个好的 RPC 框架是不能把代码写死的，所以在设计底层结构时，我会大量运用 **SPI 机制（服务发现机制）**，把它做成「插件化」的。比如未来业务想把序列化方式从 JSON 换成 Protobuf，或者想增加新的负载均衡算法，业务人员只需要写个实现类，加个配置文件就行了，一点都不需要修改我 RPC 框架的核心源码。



总结一下： 设计一个 RPC 框架，其实就是用**动态代理**做入口，把调用信息**序列化**后，包装成带有长度的**自定义协议**，再通过 Netty 发送到从**注册中心**拉取过来的、经过**负载均衡**精挑细选的远端服务器上。

让你设计一个短链系统，怎么设计？

关于短链系统的设计，这个系统在微博、营销短信里非常常见。它的核心需求其实就两件事：**一是把长长的网址变成短短的字符串，二是当用户点击短网址时，能精准地跳回原来的长链接。**

如果让我来从头设计这个系统，我会沿着整个请求的生命周期，按下面这几个核心模块去展开：

第一步是从业务上先解决「是怎么跳转的？」这个核心原理。 很多人可能觉得这是个啥黑科技，其实不管长链短链，它利用的纯粹就是 HTTP 协议里的**重定向机制**。当用户浏览器访问短链时，我们的服务器去查到对应的长链，然后给浏览器返回一个特定的状态码和一个 `Location` 响应头，浏览器就会自动跳往真实地址。



这里有个特别经典的考点：**状态码千万别用 301，要用 302。** 因为 301 是「永久重定向」，浏览器非常聪明，它第一次拿到真地址后就会把它死死缓存在本地，下次用户再点短链，浏览器自己就跳过去了，根本不经过咱们的服务器。这会导致业务方想统计个「短链点击量」完全统计不到。而 302 是「临时重定向」，每次点击都会先老老实实来请求一次咱们的服务器，方便我们做数据埋点和流量监控。

第二步也是最硬核的环节：怎么生成这个短链？ 短链一般就是类似于像 `t.cn/XyA1b2` 这种后缀大概包含 6 位随机字符（数字+大小写字母）的短串。 很多人第一时间想到用 MD5 等哈希算法对长链接做哈希处理。这种方案有个致命缺点，**哈希冲突**。哈希值如果截断成 6 位短串，很容易出现两个不同的长链接算出来一模一样的短链，处理冲突的代码会写得让人怀疑人生。



所以我更推荐目前的业界标配：「**分布式 ID 发号器 + Base62 转换**」**方案。** 这逻辑极其精妙且简单：

1. 先利用咱们常说的分布式事务 ID 生成机制（比如 MySQL 号段模式 或者 Redis 自增），给每一个新发来的长链接，分配一个**全局唯一、绝对不重复的递增数字 ID**（比如 10086）。
2. 把这个十进制的数字 ID，用类似数学里「十进制转二进制」的除法取余逻辑，转换成 **62 进制**。为什么是 62？因为 0-9、a-z、A-Z 加起来刚好 62 个字符。 您可别小看这 62 进制，哪怕只生成 **6 位长度**的短链，62 的 6 次方就能存下大概 568 亿 个不同的链接，完全够全网用好多年了，而且绝对不会发生冲突。

第三步是系统落地：怎么做存储和抗并发？ 短链系统是个典型的「**读多写少**」的系统。写操作就是生成短链，存在 MySQL 里就行，建两列，一列存数字 ID（或者生成的短串），一列存原始长链，并且给短串加上唯一索引。 但是「读操作」的并发往往极其恐怖，比如一条热点营销短信发出去，瞬间几百万点击进来查短链。所以绝对不能让请求直接打到 MySQL 上。 我们必须引入 **Redis 做缓存**。用户拿短链来访问，先去 Redis 查映射关系，查到了直接拼装 302 响应返回。查不到，再去 MySQL 捞一把，捞到了立刻写进 Redis 备用。这样系统的吞吐量就能上得去。



最后，考虑一下系统的健壮性和兜底（高级防范）。 如果是对外的公众短链服务，一定会遇到黑客恶意攻击，比如一直瞎传不存在的短链过来，试图绕过 Redis 直接把咱们的 MySQL 查穿（这就是典型的「缓存穿透」）。为了防范这一点，我会引入 **布隆过滤器（Bloom Filter）** 把所有生成过的短链存起来，或者在 Redis 里对查不到的废弃短链存一个极短时间的「空值」，直接在缓存层就把恶意请求挡回去。同时要在网关层加上 IP 的限流操作，防止被薅羊毛。



总结一下我的设计思路： 整个流转过程就是：用户请求转换长链 -> 通过**分布式 ID 发号器**拿一个唯一数字 -> 借用 Base62 转成短短的 6 位字符 -> 存入 **MySQL 和 Redis** -> 用户点击短链 -> 命中 Redis 缓存 -> 返回 **HTTP 302 状态码**完成重定向跳转。搭配好布隆过滤器防攻击，基本就能扛住大厂的日常考查了。

如何设计一个点赞系统?

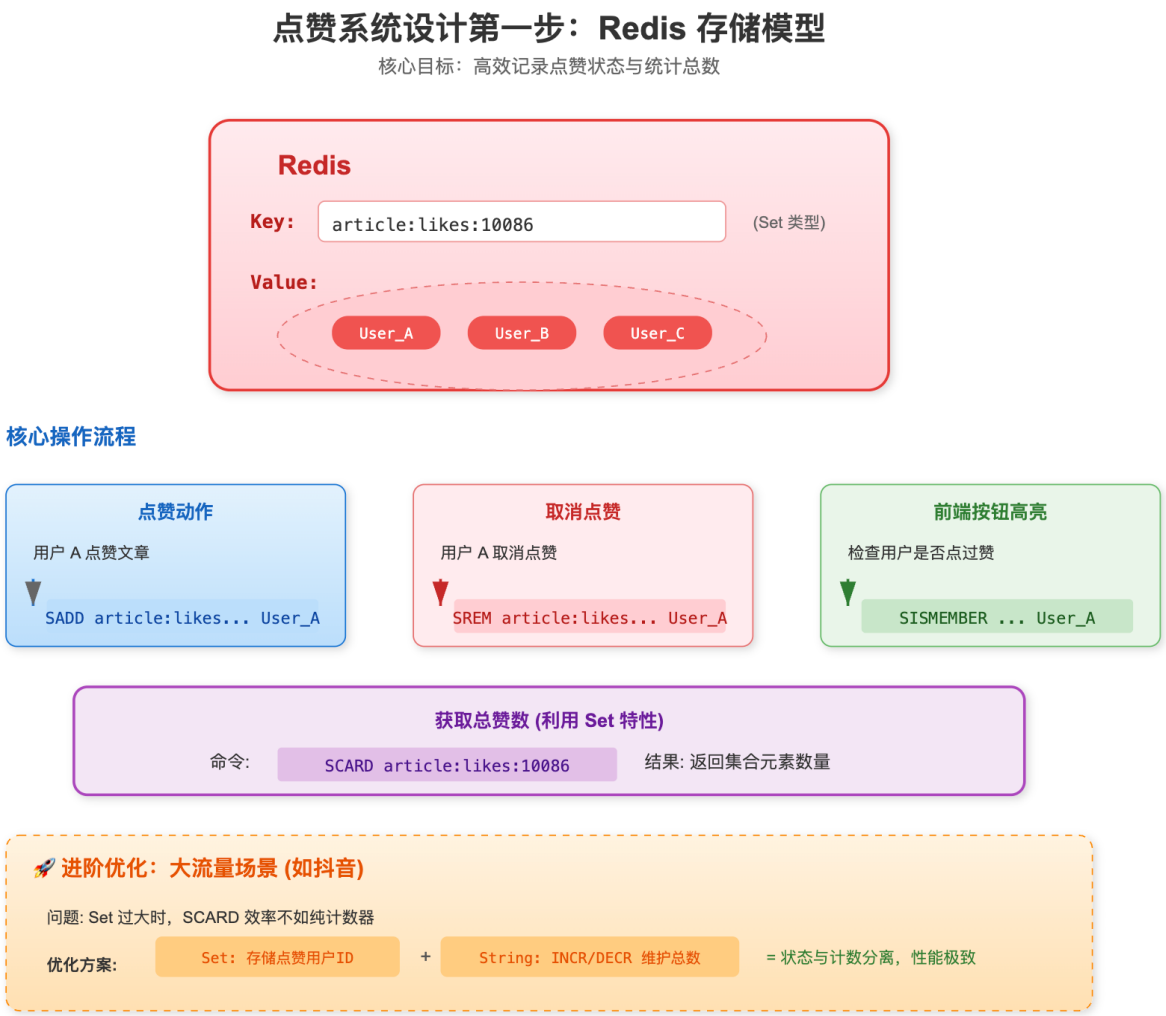
点赞这个功能看似特别简单，就点一下亮了，再点一下灭了。但在稍微大一点的互联网平台，它其实是一个**典型的「高频读写」场景**。一篇文章或一个视频发布后，可能会在短时间内涌入大量用户疯狂点赞或取消点赞，同时每一个进来刷文章的用户，系统都得立即告诉他「这篇文章总共有多少赞」以及「你刚才到底有没有点过赞」。

如果要应对这种高并发，我们传统的「直接往 MySQL 数据库里写一条记录，然后再查出来」的方案是绝对扛不住的，数据库很容易就被频繁的读写操作给打挂了。

所以，设计点赞系统的核心思路就一句话：**所有的核心读写操作必须完全交给 Redis 在内存中完成，然后再通过异步的方式悄悄把数据同步到 MySQL 做永久保存。**

具体落地上，我会把设计分为以下三个核心步骤来和您汇报：

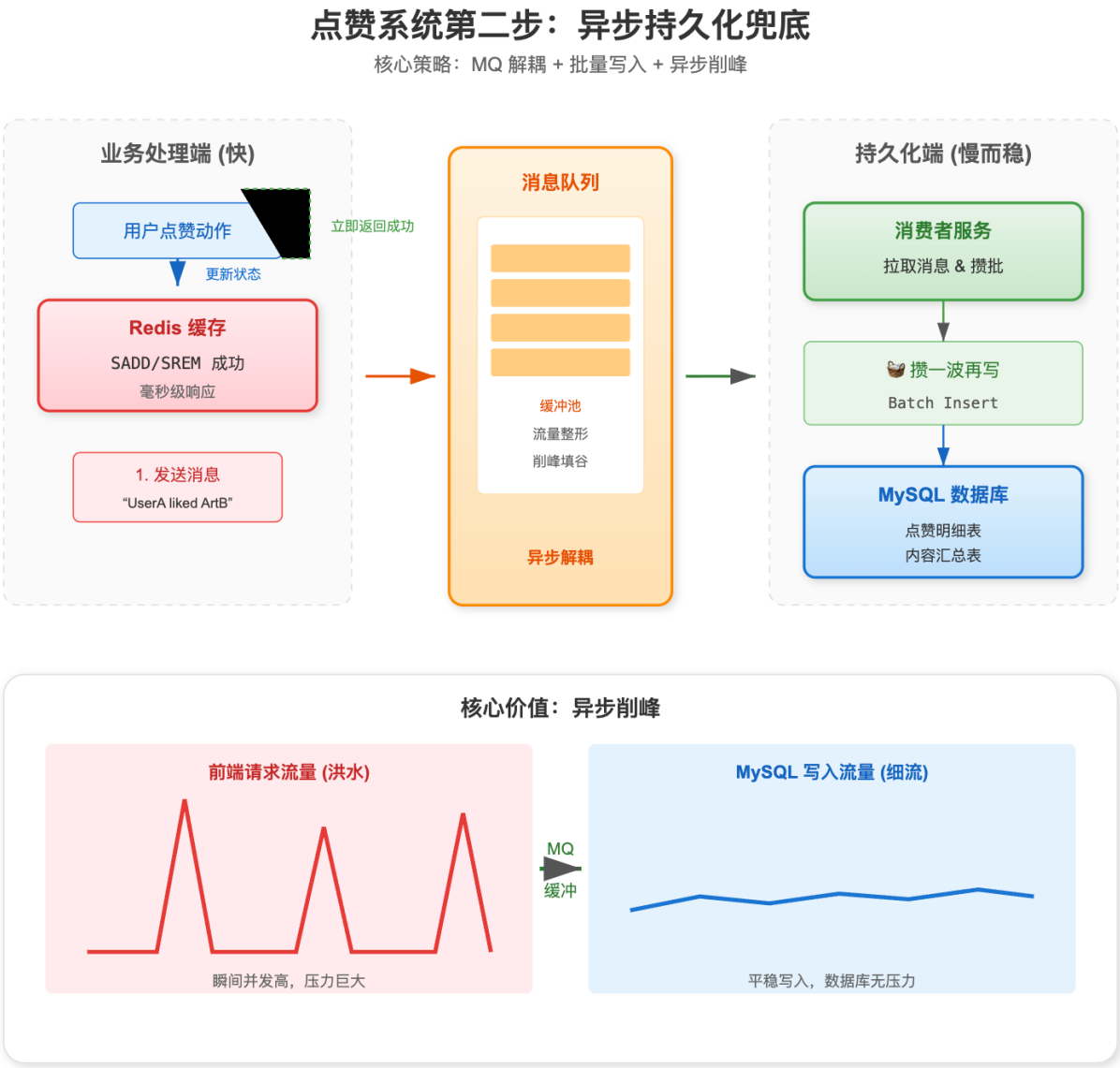
第一步是核心：在 Redis 里怎么存储点赞的数据？ 我们需要满足两个需求：获知「总赞数」和获知「某人有没有点过赞」。 对于**记录谁点过赞**，我会在 Redis 里使用 **Set（集合）** 这个数据结构。比如这篇文章的 ID 是 10086，那我就可以建一个叫 `article:likes:10086` 的 Set。



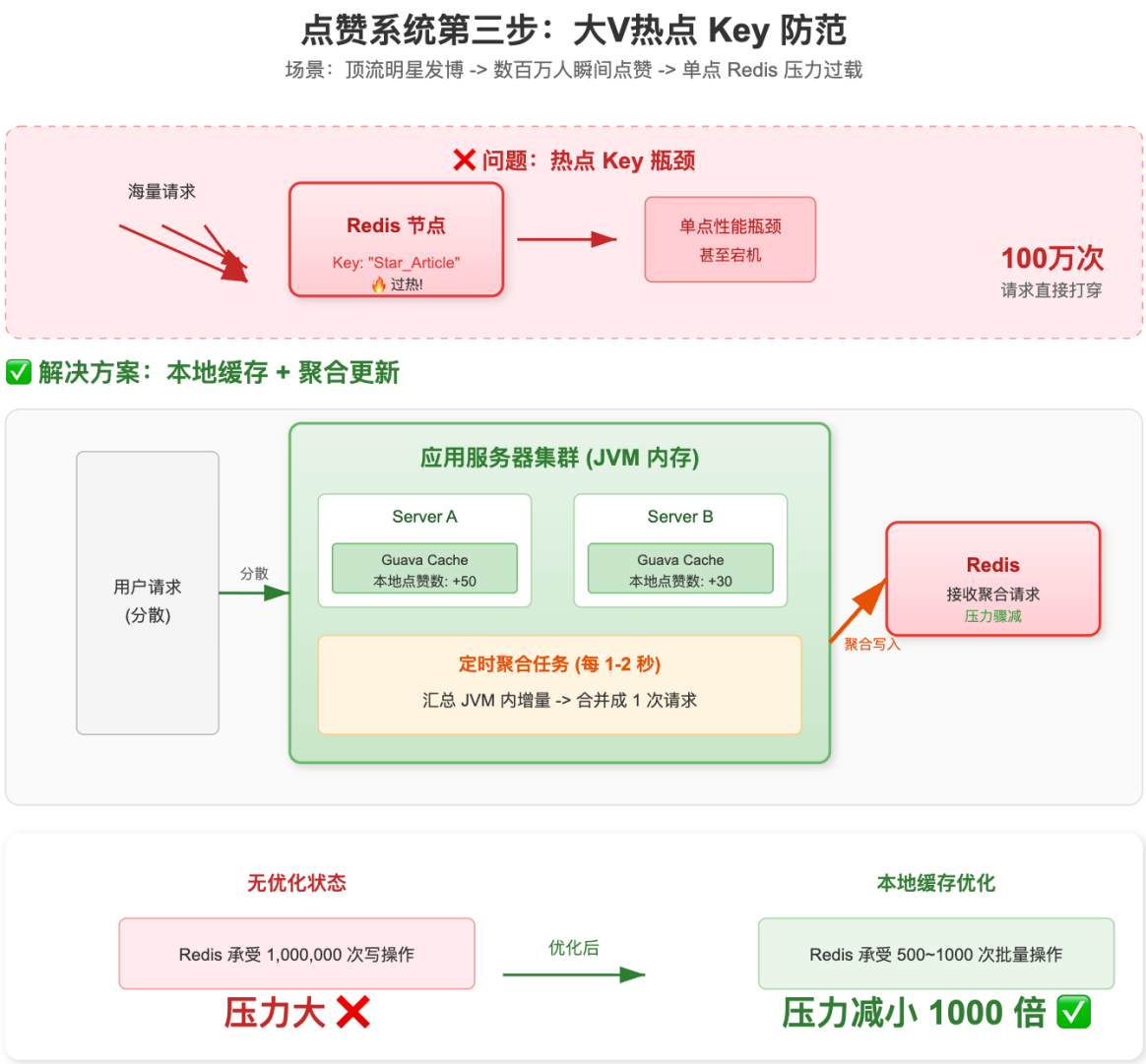
- 用户 A 点赞了，我就把他的 UserID 塞进这个集合里；
- 他取消点赞了，我就 `srem` 把他的 UserID 移出来；
- 需要高亮前端的点赞按钮时，我就用 `sismember` 判断一下他的 UserID 在不在集合里。 同时，利用 Set 自带的 `scard` 命令，我就可以非常快速地查出这个集合里有多少个人，也就是**这篇文章的「总赞数」**了。 *（补充：如果平台极大，比如抖音，可以考虑单独用 Redis 的 String 利用原子加减 `incr/decr` 来单独维护一个点赞总数，把状态和总数分开存，进一步提升性能。）*

第二步是持久化：数据怎么回到 MySQL 里兜底？ 因为 Redis 是内存数据库，如果哪怕有一点数据丢失，也是我们不想看到的，所以我们终究要把这些点赞记录存进 MySQL 的「点赞明细表」和「内容汇总表」里。 但这绝对不能是同步的。我会引入 **消息队列（MQ，比如 Kafka 或者 RocketMQ）**。 当用户在前端点赞，后端在 Redis 里操作成功后，不立刻写数据库，而是往 MQ 发

送一条消息（比如：用户 A 在 X 时间点赞了文章 B）。然后后端会有专门的消费者去慢慢拉取这些消息，**把它们攒成一波（批量操作）**，平滑、慢速地写到 MySQL 里。这就叫「**异步削峰**」，哪怕前端洪水滔天，后端数据库也能像涓涓细流一样安稳。



第三步是进阶防范：如何处理「大 V 热点」问题？ 在实际业务里，最怕的就是某位顶流明星突然发了一条微博，几百万人瞬间冲过来点赞。这时候就算全用 Redis，也可能会因为这一个帖子导致大量的请求打到同一台 Redis 服务器上，形成「**热点 Key**」问题。 为了应对这种情况，我会在 Redis 之上再加一层「**本地大对象缓存（Local Cache）**」。一旦系统探测到这是个热点文章，就把总赞数缓存进应用服务器自己的 JVM 内存里（比如用 Guava Cache），并且每隔一两秒汇总一下这段时间的「**新增赞数**」，合并成一次请求再去更新 Redis。这样就能成百上千倍地减少 Redis 的压力。



总结一下我的设计逻辑： 点赞系统的全貌就是：前端请求打过来 -> 拦截器校验后落入 **Redis 处理点赞状态（Set）和总数** -> 业务立马返回给前端响应 -> 后台通过 **消息队列（MQ）异步削峰**批量落库 MySQL -> 遇到大 V 爆款时，利用 **本地缓存汇总合并** 的手段保护系统。

让你设计一个分布式 ID 发号器，怎么设计？

我们在设计这个东西的时候，必须先明确它要达成什么目标：首先肯定得是**全局唯一**的；其次，生成的 ID 最好是「**趋势递增**」的（一般是全数字的 long 类型），因为这些 ID 最终大多数是要作为数据库主键的，如果乱序插入，会引发 MySQL 底层 B+ 树索引页的频繁分裂，导致性能剧烈下降。（*这也是为什么我们在大型商业项目里，几乎永远不会采用 UUID 做分布式主键的原因*）。

围绕着「全局唯一、趋势递增、高性能」这三点，如果让我来设计，我会提供两种业内最成熟的架构方案来应对不同的场景：

第一种方案，追求极致性能与去中心化：雪花算法（Snowflake）。 这其实是把计算逻辑全放在服务器本地的一种设计。我会把一个 64 位的数字像「切蛋糕」一样划分出不同的职能区间：

1. **最高位** 1 位空着不用。
2. 接着 **41 位用来存时间戳**（精确到毫秒），这保证了 ID 整体上是随着时间往前递增的。
3. 接着 **10 位用来存机器的专属 WorkerID**，这样哪怕多台机器同一毫秒在并发，也能保证生成的范围互不干扰。
4. 最后 **12 位留给流水号**，万一这台机器在同一毫秒内来了好几个请求，就靠这 12 位做本地的原子递增（最多一毫秒能生成 4096 个）。

这种设计的最大优势是**纯本地内存计算**，没有任何网络 IO，发号速度快得惊人。 但它有个著名的隐患叫「**时钟回拨**」，因为它极度依赖服务器本地时间。如果因为系统校时，把服务器时间往前拨慢了几毫秒，那系统就有可能回退到过去，生成出重复的 ID。针对这一点，我会在代码里做个兜底拦截，每次生成前记下上一次的时间，如果发现当前时间比上一次还小，就让线程自旋等待几毫秒，或者直接抛异常走备用发号器。



第二种方案，追求绝对高可用与紧凑排序：数据库号段模式（类似美团 Leaf 的核心思想）。 如果业务对「时钟回拨」零容忍，我就会设计一个中心化的发号服务。 一说到用数据库发号，大家直觉就是「每次来要一个 ID 就去查一次数据库，并发一高数据库就挂了」。所以「号段模式」对它进行了极其巧妙的改良，核心思想改成了「**批发**」。



简单来说：

1. 当业务服务器来请求 ID 时，发号器不给 1 个，而是直接给一个「号段」的范围，比如 `1 ~ 1000`。
2. 业务机器拿到这 1000 个号码后，就把它们**缓存在本地内存里**。接下来 1000 次用户请求，全部从应用内存里通过原子加（`AtomicLong`）悄悄分发掉，根本不需要经过网络。
3. **那快用完了怎么办？** 这里我会设计一个「**双 Buffer 机制**」。当第一批号码用到比如 20% 的时候，异步开启一个后台线程，提前去发号库把下一批号段（比如 `1001 ~ 2000`）预先加载到内存里备用。

这种设计的好处在于，它极其坚若磐石。哪怕底层的 MySQL 发号库突然宕机了十几分钟，靠着各个应用服务器内存里囤积的备用号码，整个业务系统依然能稳如泰山地运转，完全感受不到底层的故障。

总结一下我的思路： 如果是普通的互联网常规业务，希望成本足够低且没有网络依赖，我会直接引入增加防时钟回拨策略的**雪花算法**；如果是金融级别的核心链路，要求绝对可靠、ID 必须紧密递

增，我会主导搭建基于双向缓存机制的**数据库号段发号器**。实际工作中，我会根据公司的基建现状来二选一。

订单到了半个小时，半个小时未支付就取消怎么实现?

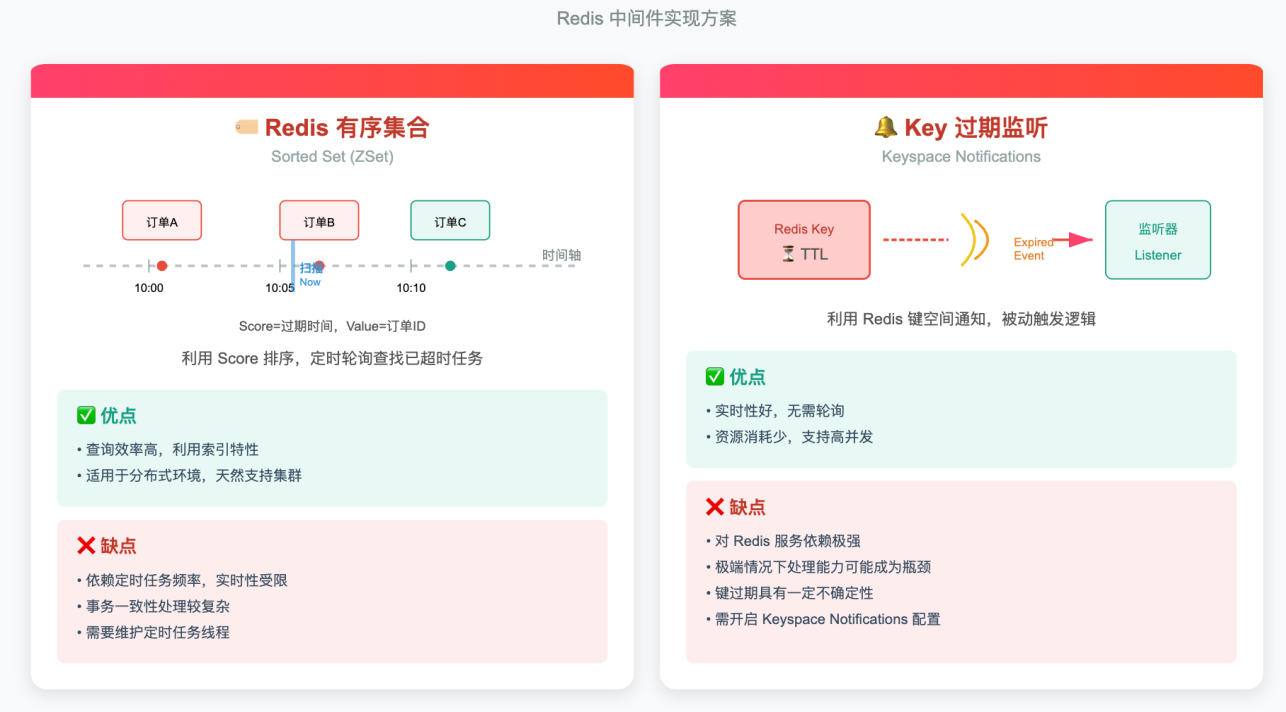
有多种实现订单超时自动取消的技术方案，包括定时轮询、JDK 的延迟队列、时间轮算法、Redis 实现以及 MQ 消息队列中的延迟队列和死信队列。

- 定时轮询：基于 SpringBoot 的 Scheduled 实现，通过定时任务扫描数据库中的订单。优点是实现简单直接，但缺点是会给数据库带来持续压力，处理效率受任务执行间隔影响较大，且在高并发场景下可能引发并发问题和资源浪费。
- JDK 的延迟队列（DelayQueue）：基于优先级队列实现，减少数据库访问，提供高效的任务处理。优点是内部数据结构高效，线程安全。缺点是所有待处理订单需保留在内存中，可能导致内存消耗大，且无持久化机制，系统崩溃时可能丢失数据。
- 时间轮算法：通过时间轮结构实现定时任务调度，能高效处理大量定时任务，提供精确的超时控制。优点是实现简单，执行效率高，且有成熟实现库。缺点同样是内存占用和崩溃时数据丢失的问题。



- Redis 实现：

- 有序集合（Sorted Set）：利用有序集合的特性，定时轮询查找已超时的任务。优点是查询效率高，适用于分布式环境，减少数据库压力。缺点是依赖定时任务执行频率，处理超时订单的实时性受限，且在处理事务一致性方面需要额外努力。
- Key 过期监听：利用 Redis 键过期事件自动触发订单取消逻辑。优点是实时性好，资源消耗少，支持高并发。缺点是对 Redis 服务的依赖性强，极端情况下处理能力可能成为瓶颈，且键过期有一定的不确定性。



- MQ 消息队列：
  - 延迟队列（如 RabbitMQ 的 rabbitmq_delayed_message_exchange 插件）：实现消息在指定延迟后送达处理队列。优点是处理高效，异步执行，易于扩展，模块化程度高。缺点是高度依赖消息队列服务，配置复杂度增加，可能涉及消息丢失或延迟风险，以及消息队列与数据库操作一致性问题。
  - 死信队列：通过设置队列 TTL 将超时订单转为死信，由监听死信队列的消费者处理。优点是能捕获并隔离异常消息，实现业务逻辑分离，资源保护良好，方便追踪和分析问题。缺点是相比延迟队列，处理超时不够精确，配置复杂，且同样存在消息处理完整性及一致性问题。



不同方案各有优劣，实际应用中应根据系统的具体需求、资源状况以及技术栈等因素综合评估，选择最适合的方案。在许多现代大型系统中，通常会选择消息队列的延迟队列或死信队列方案，以充分利用其异步处理、资源优化和扩展性方面的优势。

如果你的系统的 QPS 突然提升 10 倍你会怎么设计?

如果真的遇到这种情况，我会从「**紧急止血（保命）**」和「**架构全面演进（治本）**」两个阶段来设计和应对。

首先是第一阶段：紧急应对，核心原则是「弃车保帅，保证系统不挂」。 如果是突发流量，系统可能瞬间被打死，所以第一时间一定是**限流和降级**。 我们可以利用像 Sentinel 这样的工具，在网关层就设置好最大 TPS 阈值，超过系统承载能力的请求直接快速失败，返回「系统繁忙」的提示。即使损失了一部分用户的体验，也绝对比整个系统崩溃、所有人都不能用要强。 另外就是**业务降级**，比如在电商场景下，我会立刻关掉像「猜你喜欢」、评论显示、积分发放这些非核心业务，把所有的服务器 CPU 和内存资源全让给「下单」和「支付」这两个最核心的动作。如果公司的基建完善，我会马上通过 K8s 动态增加容器节点来进行**弹性扩容**，用加机器来硬扛。



接下来是第二阶段：如果这 10 倍 QPS 变成了常态，我们的架构必须要进行彻底的演进。 这就需要把系统内部拆开来看，把读流量和写流量分开治理。

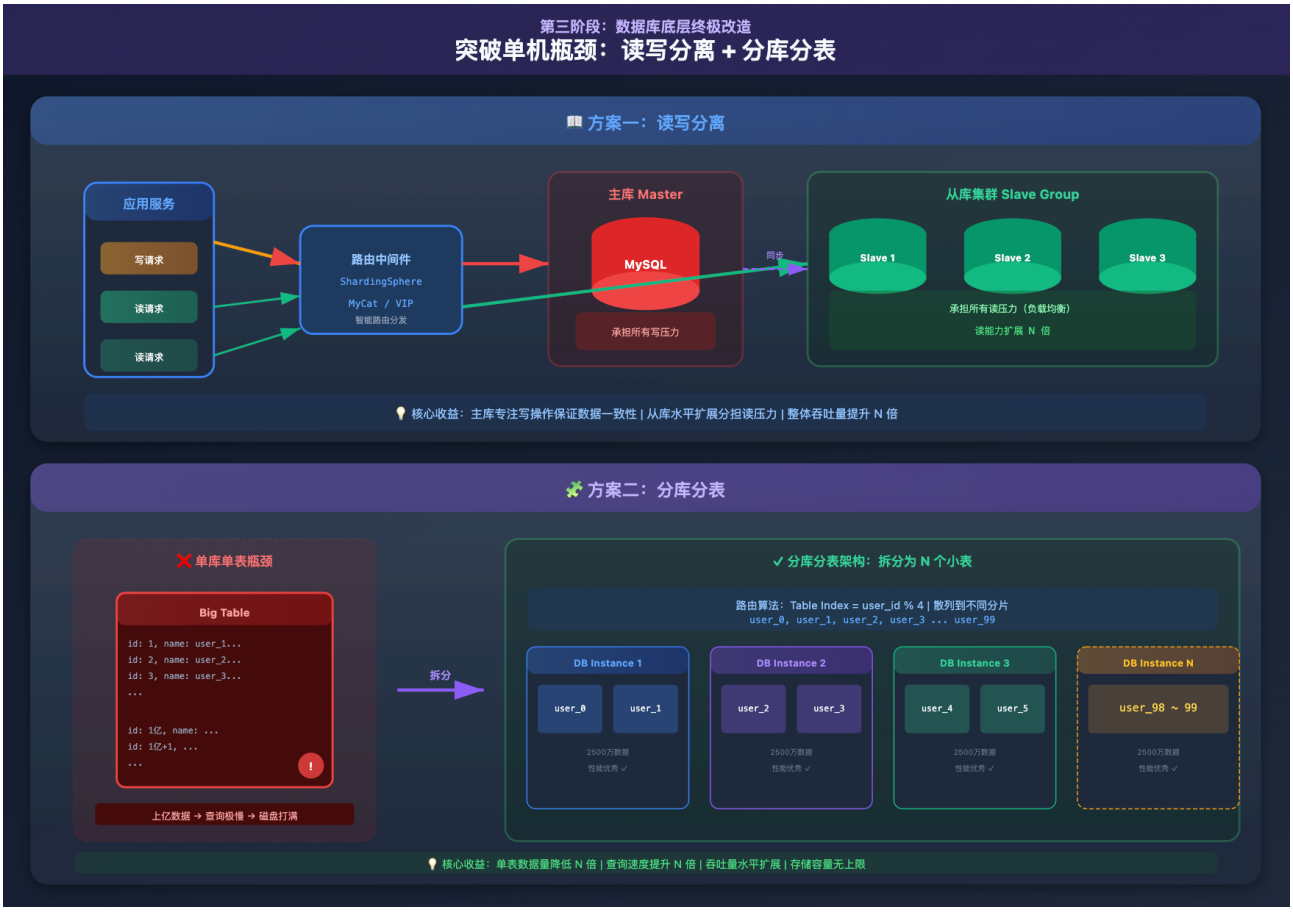
- **第一，要想扛住 10 倍的读请求，唯一的法宝就是「多级缓存储备」。** 这个时候单靠底层的 MySQL 是绝对不可能的，哪怕只靠 Redis 集群可能也会因为网络带宽被打满而出现瓶颈。所以，我一定会引入「**本地缓存（比如 Caffeine 或 Guava） + Redis 分布式系统缓存**」的双层架构。 把那些极少变化的热点数据，直接加载到应用服务器自己的 JVM 内存里。这样大量的读请求连网络请求都不用发，在服务器本地就直接拿到数据返回了，系统的响应时间会被压到极致，也能极大地保护后端的 Redis 和 MySQL。



- **第二，要想扛住 10 倍的写请求，核心思路是「排队削峰」。** 当 10 倍的用户同时进行下单或者点赞这种写操作时，绝对不能让请求直接打进数据库。我会引入 **消息队列（MQ，比如 Kafka 或 RocketMQ）**。 前台的请求一过来，只要基础校验通过，立刻扔一条消息进 MQ，然后马上告诉前端「请求正在处理中」。接着，后端的服务根据数据库能承受的真实写入速度，平稳、慢速地从 MQ 里拉取请求去执行。这就像游乐场门口的蛇形排队通道一样，把瞬间爆发的流量洪峰，变成了平平稳稳的细水长流。



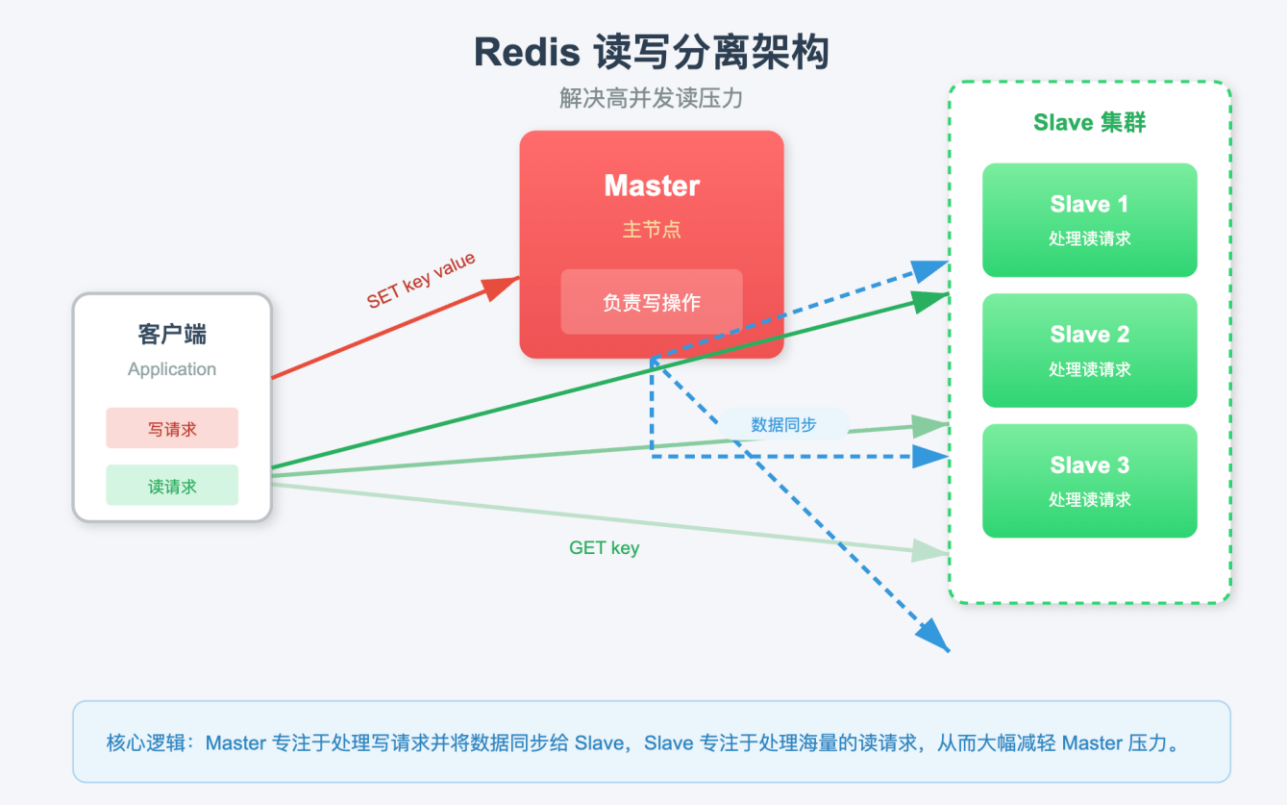
- **第三，数据库底层的终极改造。** 如果上述方案做完，发现底层存储的数据量和并发连接数还是达到了瓶颈，那说明单台 MySQL 彻底到极限了。这时候就必须走向「**读写分离**」和「**分库分表**」。用主库抗写压力，多台从库抗读压力；把原来的一张上亿级的大表，按规律拆分成 100 张小表，分散到不同的机器上，以此来成倍扩充底层的吞吐量。



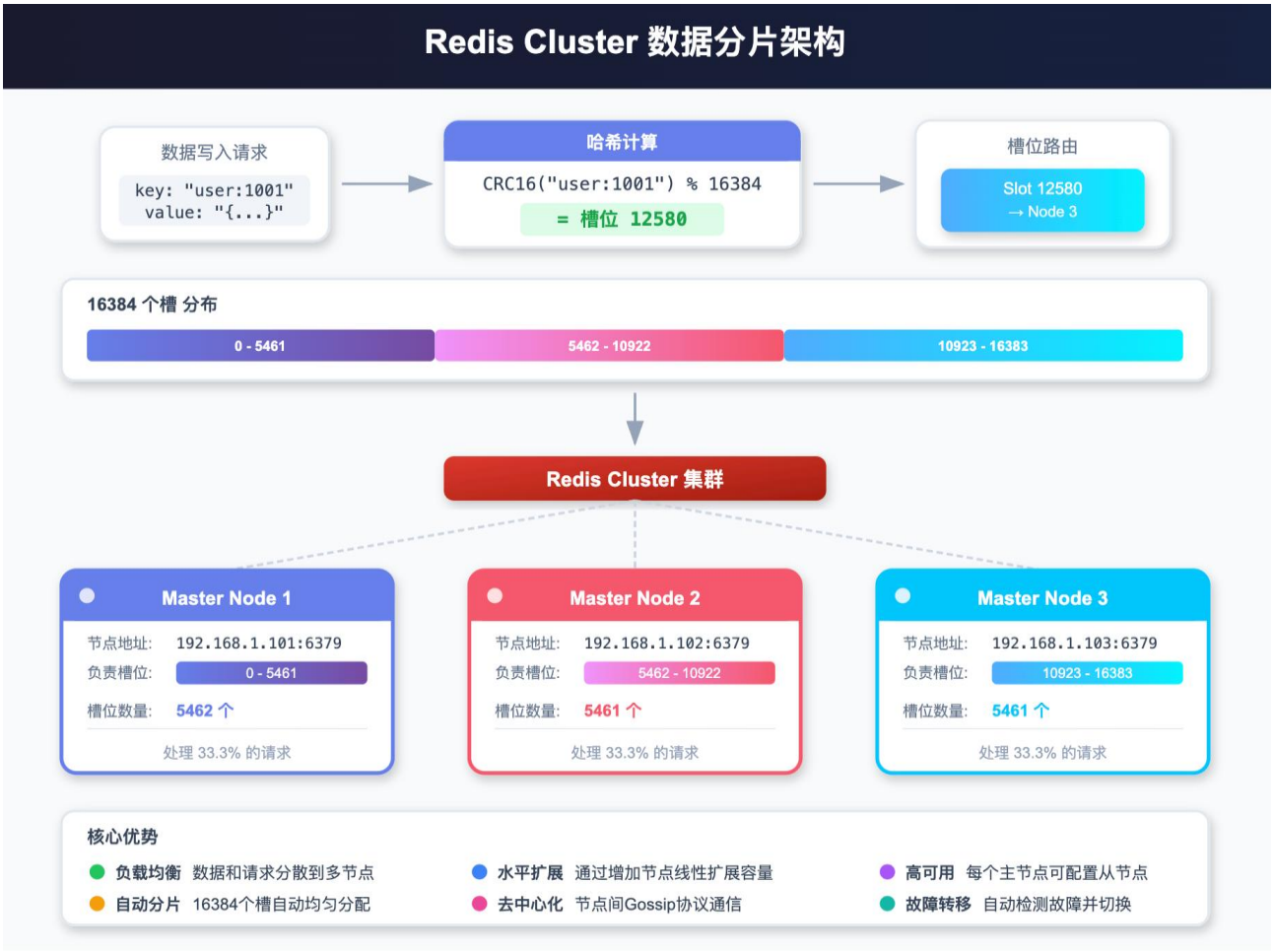
总结一下我的完整思路就是： 面对 10 倍激增，先用**限流、降级和扩容**保住系统的命；然后用**多级客户端及本地缓存**挡住绝大部分读流量；用 **MQ 异步削峰**消化突发的写流量；最后兜底改造数据库，推进**分库分表**。一套组合拳下来，基本就能把这 10 倍的 QPS 稳稳地吃下来了。

如果做一个大流量的网站，单 Redis 无法承压了如何解决?

- 读写分离：部署多个 Redis 从节点（Slave），主节点（Master）负责写操作，从节点负责读操作。主节点将数据同步到从节点，从节点可以处理大量的读请求，减轻主节点的压力。



- 构建集群：部署 Redis Cluster 集群，Redis Cluster 将数据自动划分为 16384 个槽（slots），每个槽都可以存储键值对。这些槽会被分配到多个 Redis 节点上，通过哈希函数将键映射到相应的槽，再由槽映射到具体的 Redis 节点。例如，使用 `CRC16(key) % 16384` 来确定键属于哪个槽，然后根据槽与节点的映射关系将键值对存储到相应节点。通过数据分片，将数据和请求分散到多个节点，避免单个节点的负载过高。不同节点负责不同的槽，各自处理一部分请求，实现负载均衡。



如何设计一个可重入的分布式锁，用什么结构设计？

可重入锁允许同一线程在持有锁的情况下多次获得锁而不会产生死锁。当线程请求锁时，如果它已经拥有了该锁，则可以直接获得锁，锁的计数器会增加。在释放锁时，计数器会减少，只有当计数器为零时，锁才会真正释放。

在分布式系统中，通常会使用诸如 Redis、Zookeeper 或 etcd 等组件来实现锁的管理。下面以 Redis 为例，简单描述如何设计一个可重入的分布式锁。

设计要素：

- **锁的标识符（Lock ID）**：用于标识锁的唯一性。
- **持有者标识（Owner ID）**：线程或进程的唯一标识符，通常是线程的 ID 或进程的 ID。
- **计数器**：记录当前持有锁的次数。
- **过期时间**：为了避免死锁，锁需要设定一个合理的超时时间。

我们可以在 Redis 中使用一个哈希表或简单的键值对来存储锁的状态。使用一个 key（如 `lock:<resource>`）来表示锁，包含以下字段：

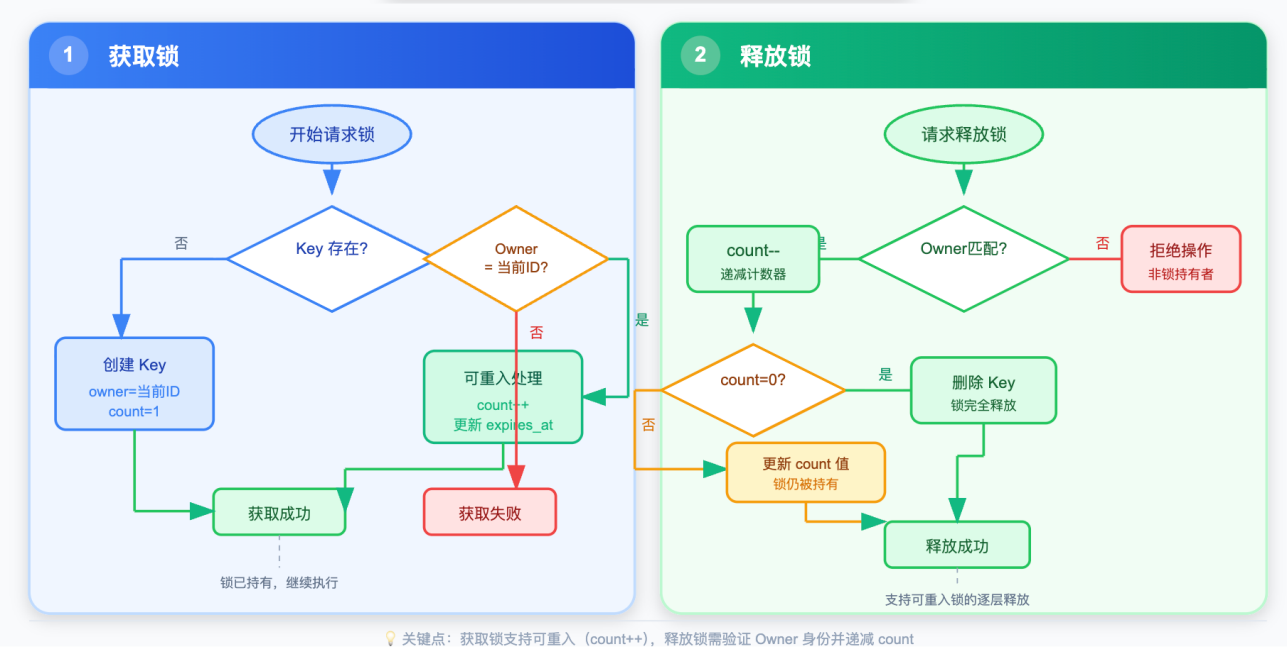
- `owner`: 当前持有锁的线程/进程标识符

- `count`：当前计数器，表示获得锁的次数
- `expires_at`：锁的到期时间，用于防止死锁

示例数据结构

```
{
  "key": "lock:resource_1",
  "value": {
    "owner": "thread_id_or_process_id",
    "count": 3,
    "expires_at": "2023-10-01T12:00:00Z"
  }
}
```

以下是围绕这个锁结构的基本操作：



1. 获取锁 (Lock Acquisition)：尝试设置 lock: 的值，如果这个 key 不存在（即没有锁），那么可以创建该 key，并设置 owner 为当前线程/进程的唯一 ID，count 设为 1，expires_at 设为当前时间加上锁的过期时间。 如果该 key 已存在且 owner 是当前线程/进程的 ID，则增加 count 并更新 expires_at。
2. 释放锁 (Lock Release)：检查当前的 owner 是否是当前线程/进程的 ID。如果是，减少 count。如果 count 减少到 0，则删除该 key。 如果在持有锁的情况下，锁已过期，系统会根据 expires_at 检查锁是否可释放，避免死锁。

3. 锁续期 (Lock Renewal)：如果当前线程在执行过程中需要继续持有锁，可以在逻辑处理中重新设置 expires_at.
4. 超时与故障恢复：可以设置锁的自动超时时间，例如，设定一个最大持锁时间，超时后锁自动释放。这可以在一定程度上防止死锁的情况。

整体流程示例：

- **获取锁：**
  - 线程 A 调用获取锁的 API。如果成功，返回锁的状态。
  - 如果线程 B 也尝试获取同一把锁，则会返回锁已被占用。
- **释放锁：**
  - 线程 A 在完成任务时调用释放锁的 API，递减 `count` 字段。
  - 如果 `count` 达到 0，删除锁并释放资源。
- **续约锁：**
  - 线程 A 在处理过程中可以根据需要续约锁，更新 `expires_at`。

你有看过一些负载均衡的一些方案吗？

- **硬件负载均衡**：使用专用的硬件设备（如负载均衡器）来分配流量，比如 F5 设备，优点是性能强大，支持高并发。缺点是成本太高，通常一个专业级的硬件设备，都需要百万级别的价格。
- **软件负载均衡**：使用软件应用程序来实现负载均衡，可以运行在普通服务器上，比如 **Nginx** 支持反向代理和负载均衡，优点灵活性高，成本低，易于修改和扩展，缺点是性能不如硬件负载均衡，但是软件负载均衡的性能也足够应对大多数场景的并发量了。
- **DNS 负载均衡**：通过 DNS 服务器将流量分配到多个服务器上，不同的客户端请求可能获得不同的 IP 地址。优点是简单易用，不需要额外的硬件或软件。缺点是无法智能感知服务器的实时状态，缓存问题可能导致不均匀负载。
- **内容分发网络 (CDN)**：CDN 是一种分布式的网络结构，可以将内容分发到离用户最近的节点上。优点减少延迟，提高用户体验。缺点主要适用于静态内容，动态请求仍需其他形式的负载均衡。

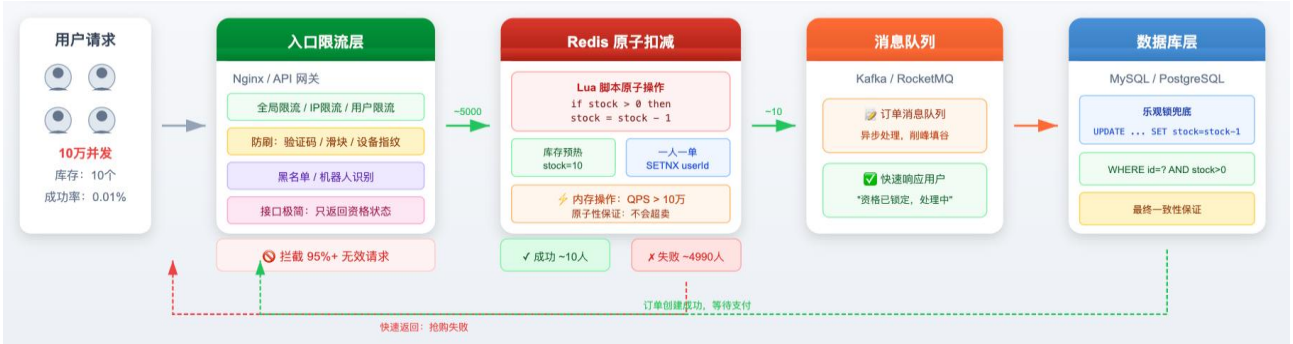


10w 顾客抢购一个只有 10 个库存的商品如何设计?

这个问题其实就是典型的「秒杀」场景。核心目标其实就两个：

- 第一，系统别被 10 万并发直接打崩；
- 第二，库存绝对不能超卖，用户体验也别太差。

我的整体思路是分三层：**先在入口把流量「削平」，再在中间用一个足够快的组件做原子扣减，最后把下单这种重活异步化落库**。



首先入口层我一定会做限流和隔离。像 Nginx / 网关层做全局限流、按 IP/用户限流，防刷和机器人校验也要上，比如验证码、滑块、设备指纹之类的。然后接口本身要做到极简，只干一件事：拿资格。因为如果每个请求都直接打到数据库去查库存，10 万并发数据库必死。

然后真正防超卖的关键我会放在 Redis 或者类似的原子计数里做。最常见的做法是把库存预热到 Redis，用 Lua 脚本做「检查库存>0 并扣减」的原子操作，同时做一人一单的校验，比如用 `SETNX userId` 或者在 Lua 里顺便判断用户是否已抢过。这样一个请求过来，要么直接返回成功拿到资格，要么失败，整个过程完全在 Redis 内完成，性能扛得住，而且原子性保证不会超卖。

拿到资格之后我不会立刻同步走创建订单、扣数据库库存这种重流程，而是把它写进消息队列，比如 Kafka/RocketMQ。这样前端能很快收到「你已抢到资格，订单处理中」的结果，系统后面慢慢消费消息去真正创建订单、落库、扣真实库存。

数据库层我会用「最终一致」的方式兜底：比如库存表用乐观锁 `update ... set stock=stock-1 where id=? and stock>0`，就算 Redis 那边有极端情况，也能在 DB 这层再挡一次超卖。

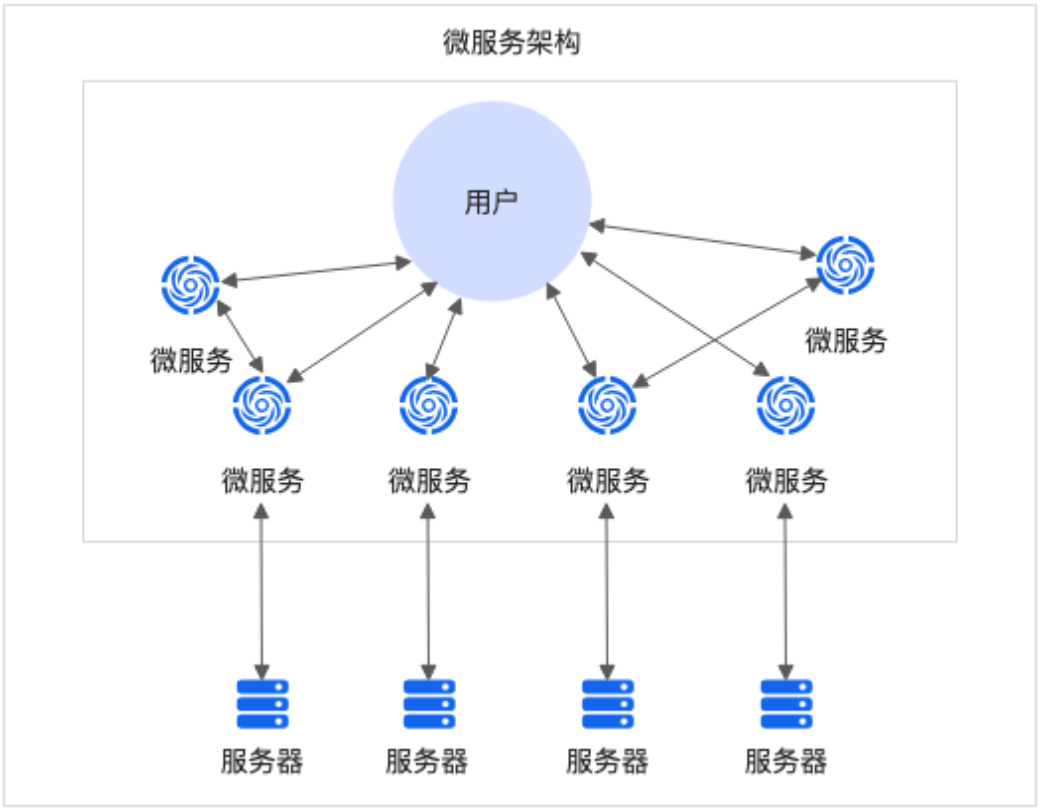
再往后，我会补两个关键的工程细节。

- 一个是「订单有效期」，因为抢到资格的人不一定付款，所以要给订单一个超时时间，比如 15 分钟不支付就自动取消，并把库存归还到 Redis 和 DB，这块也通常靠延迟消息或者定时任务做。
- 另一个是「重复请求和幂等」，因为用户会狂点、网络会重试，所以资格发放和下单消费都要有幂等键，比如以 userId+skuId 作为唯一键，保证不会重复创建订单。

总结一下就是：入口限流抗压，Redis 原子扣减防超卖，MQ 异步落库抗峰值，DB 乐观锁兜底一致性，再加上超时释放库存和幂等，基本就能把 10 万抢 10 个这种场景稳住。

微服务架构有了解吗？你认为什么是微服务架构？

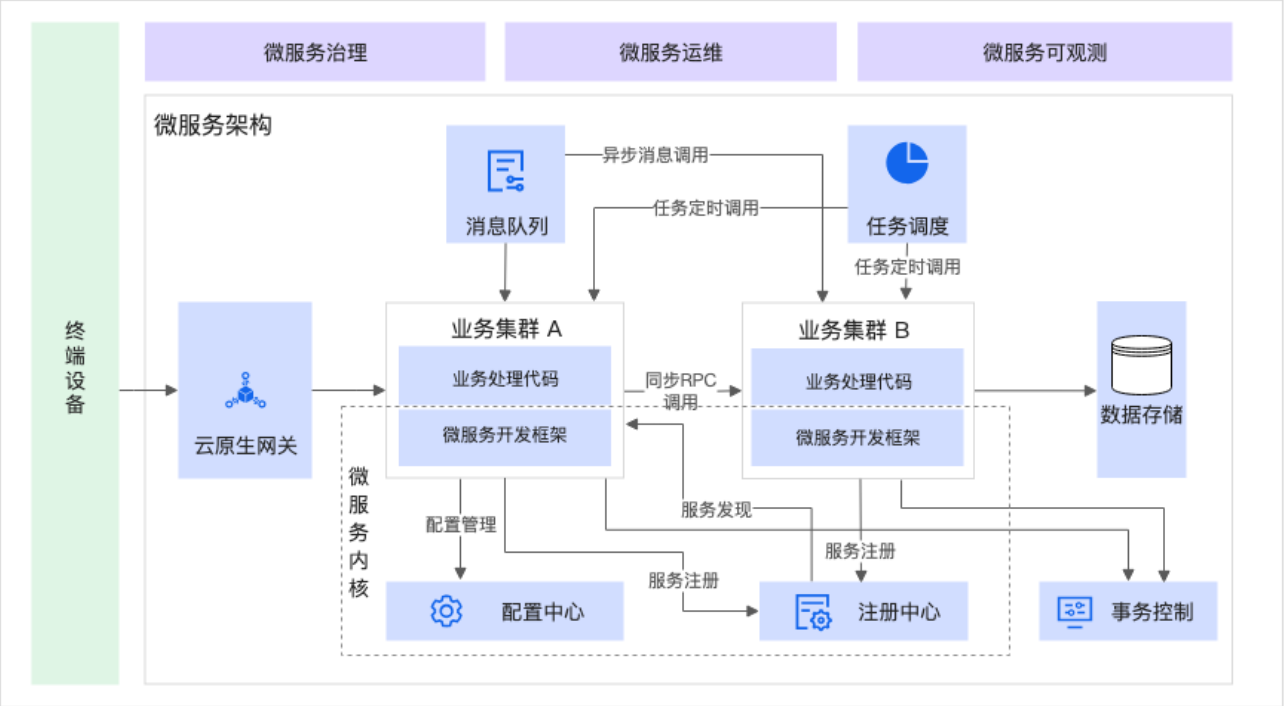
简单说微服务就是把原来一个大的单体应用拆分成多个小的独立服务，每个服务负责一块具体的业务功能，可以独立开发、独立部署、独立扩展。



微服务最大的好处就是灵活。比如电商系统，我们可以把用户服务、订单服务、商品服务、支付服务都拆开，如果双十一订单量暴增，我只需要把订单服务多部署几个实例就行，不用整个系统都扩容，这样既省资源又快。而且不同团队可以并行开发不同的服务，用自己熟悉的技术栈，发版也互不影响，开发效率会高很多。



但微服务也不是银弹，它带来的复杂度其实挺高的。首先服务之间要互相调用，这就需要服务注册发现，我们一般用 Nacos 或者 Eureka 这种注册中心，让服务启动时把自己注册上去，调用方从注册中心拉取服务列表。然后调用方式上可以用 HTTP 的 RESTful 或者 RPC 框架像 Dubbo、gRPC，这些都要考虑负载均衡、超时重试、熔断降级这些容错机制。



链路追踪也很重要，一个请求可能要经过五六个服务才完成，出了问题不好排查，所以我们会用 Skywalking 或者 Zipkin 把整个调用链路串起来，每个请求都带一个 TraceID，方便追溯。配置管理也是个问题，微服务多了之后配置文件一大堆，我们用配置中心统一管理，修改配置可以动态下发不用重启服务。

网关是微服务的入口，像 Spring Cloud Gateway 或者 Nginx，负责路由转发、鉴权限流这些统一处理的逻辑。服务间通信如果都是同步调用，一个服务慢了会拖累整条链路，所以有些场景我们会用消息队列做异步解耦，比如下单后发个消息让库存服务去扣减，不用同步等待。

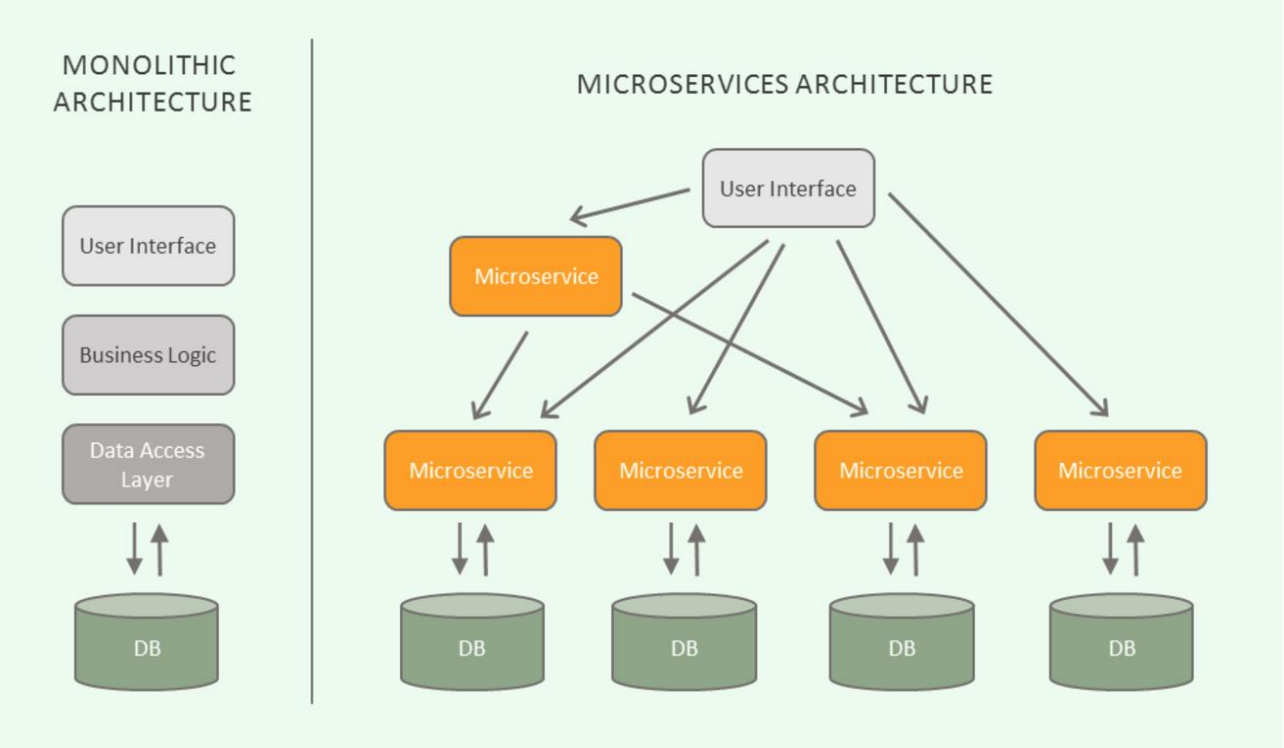
数据管理也比较头疼，微服务一般提倡每个服务有自己的数据库，不能直接访问别人的表，这样能做到服务自治。但这也带来分布式事务的问题，一个业务操作涉及多个服务的数据修改，要保证一致性就得用 Seata 这种分布式事务框架，或者用最终一致性方案像 TCC、SAGA 模式。

我觉得微服务适合业务复杂、团队规模大、需要快速迭代的场景。如果就是个小项目，团队就三五个人，其实单体应用更合适，不要为了微服务而微服务。架构选型一定要结合实际情况，微服务能解决很多问题，但也会引入新的复杂度，这个要权衡好。

对于你的项目（单体），怎么拆分成微服务架构呢？

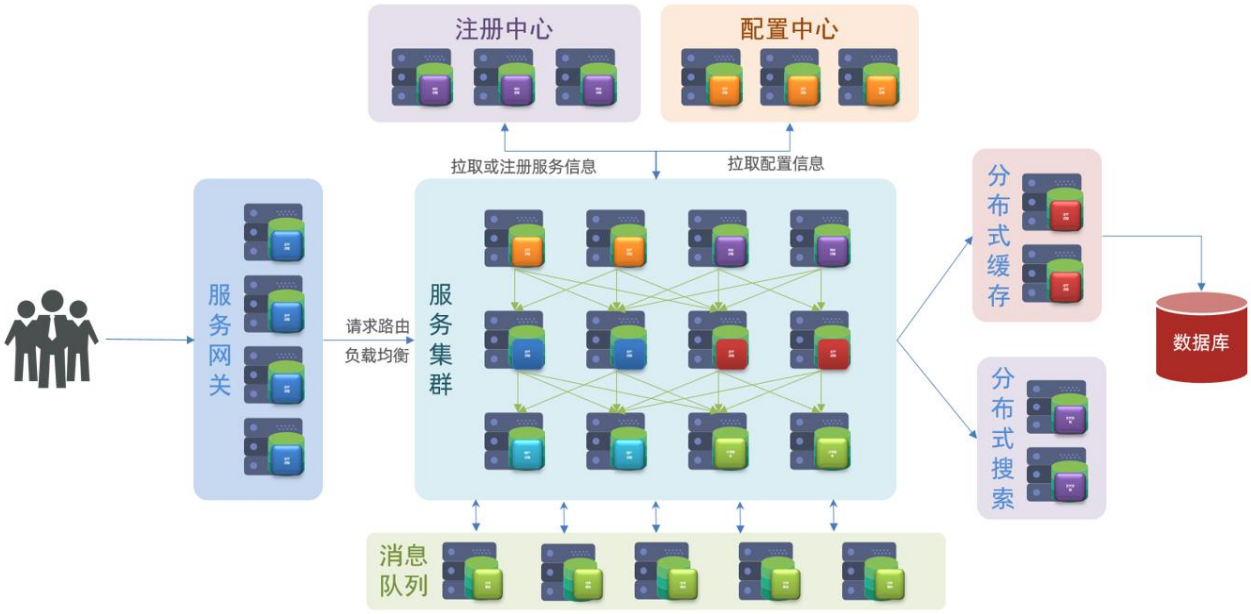
拆分微服务我觉得不能一上来就全拆，得循序渐进。首先我会梳理现有系统的业务模块和依赖关系，画个模块依赖图，看哪些模块相对独立、哪些耦合比较重。

然后按照业务领域来划分服务边界，遵循领域驱动设计的思想。比如电商系统，用户、商品、订单、支付、库存这些是不同的业务领域，职责比较清晰。我会优先拆那些业务独立、变更频繁或者需要单独扩展的模块，比如支付模块调用量大、对安全性要求高，就很适合先独立出来。



拆的时候我会用绞杀者模式，新功能用微服务开发，老功能逐步迁移。先拆一个相对简单的模块试水，把注册中心、配置中心、网关、监控这些基础设施搭起来，等第一个服务跑稳了再陆续拆其他模块，这样风险可控。

数据库拆分是个难点。我会先做逻辑拆分，不同服务访问不同的 schema 或表，但物理上还在一个库。等服务边界稳定了再做物理拆分，把数据迁到各自的数据库。原来用事务能解决的一致性问题，拆开后可能要用分布式事务或者最终一致性方案，有些强一致的场景我会考虑暂时不拆。



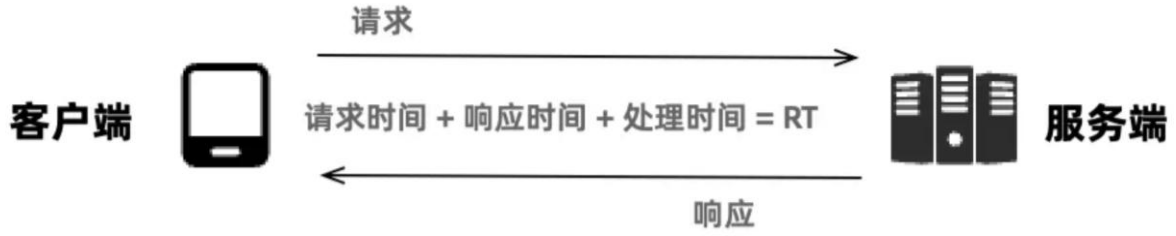
服务间调用方面，原来直接方法调用，拆开后要走网络，我会用 Dubbo 或 Feign 做远程调用，加上超时控制、熔断降级。如果调用链路长，还可以用消息队列做异步解耦。

公共代码要么抽成 SDK 让各服务依赖，要么单独做成基础服务。监控和日志也要升级，用 ELK 做日志收集，加上链路追踪方便排查问题。

整个过程我会保持小步快跑，每拆一个服务都充分测试，做好灰度发布和回滚预案。我觉得拆分核心是业务驱动，真正能解决单体应用的扩展性和开发效率问题，而不是为了拆而拆。

对外提供一个 api 服务，客户说请求接口超时了，怎么排查?

我第一步会先问清楚基本情况，是偶发还是持续超时？能不能提供请求 ID 或者具体时间点？是所有客户还是个别客户？这些信息能帮我快速判断问题范围。

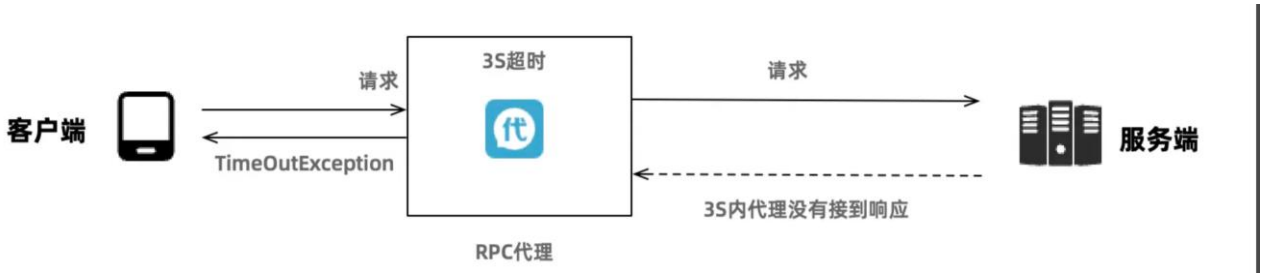


然后我会立刻看监控大盘，看接口的响应时间、QPS、错误率有没有异常，同时看服务器的 CPU、内存、网络 IO 这些基础指标有没有打满。如果整体都慢那可能是系统级问题，如果只是个别请求慢就要具体分析。

接下来我会带着请求 ID 去查日志，看请求链路上哪个环节慢了。是数据库查询慢？还是调用下游服务慢？还是某个业务逻辑耗时长？如果用了分布式追踪工具像 Skywalking，能更直观看到每一跳的耗时。

数据库这块我会重点关注，查一下慢查询日志，看有没有 SQL 走错索引或者连接池满了。如果用了 Redis 缓存，还要看缓存命中率有没有下降，缓存失效可能导致大量请求打到数据库。

下游依赖也是重灾区，我会确认调用的其他服务有没有故障或者变更。网络层面也要排查，看有没有丢包、DNS 解析慢、或者跨地域网络抖动这些问题。



如果最近有过发布，我会特别注意是不是新代码引入的问题，比如加了耗时操作或者配置改错了。还要看 JVM 有没有频繁 Full GC，容器有没有被限流。

找到原因后我会快速处理，该加索引加索引，该扩容扩容，该熔断降级就降级，优先保证服务恢复。处理完后我会做复盘，加监控告警，完善降级策略，避免再次发生。

我觉得排查超时的核心是从全局到局部，先看监控定范围，再查日志找细节，同时要关注数据库、下游依赖、网络、代码变更这几个高频问题点，这样能更快定位问题。